

Министерство науки и высшего образования Российской Федерации  
Федеральное государственное бюджетное образовательное учреждение высшего  
образования

**«Кубанский государственный университет» (КубГУ)**

Факультет прикладной математики

Кафедра математического моделирования

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА  
(МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ)**

**СОВМЕСТНОЕ АДАПТИВНОЕ СПЕКУЛЯТИВНОЕ  
ДЕКОДИРОВАНИЕ  
БОЛЬШИХ ЯЗЫКОВЫХ МОДЕЛЕЙ НА ОСНОВЕ МАРКОВСКОГО  
ПРОЦЕССА ПРИНЯТИЯ РЕШЕНИЙ (JOINTADASPEC)**

Работу выполнил \_\_\_\_\_ А.А. Козин

Направление подготовки 01.04.02 Прикладная математика и информатика

Направленность (профиль) Математическое и программное обеспечение  
вычислительных машин и систем

Научный руководитель \_\_\_\_\_

Краснодар 2026

## РЕФЕРАТ

Выпускная квалификационная работа (магистерская диссертация): 78 с., 4 раздела, 10 рис., 10 табл., 19 источников, 4 приложения.

СПЕКУЛЯТИВНОЕ ДЕКОДИРОВАНИЕ, БОЛЬШИЕ ЯЗЫКОВЫЕ МОДЕЛИ, УСКОРЕНИЕ ИНФЕРЕНСА, МАРКОВСКИЙ ПРОЦЕСС ПРИНЯТИЯ РЕШЕНИЙ, VALUE ITERATION, АДАПТИВНОЕ УПРАВЛЕНИЕ, ДЛИНА ЧЕРНОВИКА, ПОРОГ ВЕРИФИКАЦИИ, JOINTADASPEC.

Объектом исследования является процесс инференса авторегрессионных больших языковых моделей, использующих спекулятивное декодирование.

Предметом исследования являются методы совместной адаптивной оптимизации длины черновика и порога верификации в спекулятивном декодировании.

Цель работы — разработка и экспериментальное исследование метода адаптивного спекулятивного декодирования, который совместно управляет длиной черновика и порогом нечёткой верификации для повышения пропускной способности инференса при контролируемом качестве генерации.

Методы исследования: формализация управления в виде табличного марковского процесса принятия решений, решение методом разреженной итерации по ценности (value iteration), парный критерий Макнемара и бутстреп-оценка 95 %-х доверительных интервалов при сравнении методов.

Основные результаты. Предложен метод JointAdaSpec, формализующий совместное управление двумя осями спекулятивного декодирования как задачу оптимального управления. На паре моделей Qwen2.5-14B-Instruct → Qwen2.5-0.5B-Instruct получен статистически значимый прирост точности +4,07 п.п. exact match на наборе GSM8K ( $p = 0,0205$ , критерий Макнемара,  $n = 1500$  парных наблюдений) при пропускной способности 10,84 ток/с, что в 2,22 раза выше ванильного спекулятивного декодирования. Доказано, что совместная и каскадная политики статистически неразличимы ( $\Delta = -0,13$  п.п.,  $p = 0,96$ ), что

является точным следствием доказанной теоремы о value gap. На паре с низким соотношением мощностей ( $7B/1,5B$ ,  $4,7\times$ ) прирост качества не подтверждён — установлено граничное условие применимости метода.

Научная новизна состоит в том, что впервые длина черновика и порог нечёткой верификации рассматриваются как связанные переменные управления в единой оптимизационной постановке, для которой получены аналитические гарантии корректности и субоптимальности каскадных эвристик.

Практическая значимость определяется тем, что метод восстанавливает пропускную способность, теряемую ванильным спекулятивным декодированием, без обучения дополнительных нейросетевых модулей и реализован в виде воспроизводимого программного комплекса с открытым исходным кодом.

## СОДЕРЖАНИЕ

|                                                                                          |           |
|------------------------------------------------------------------------------------------|-----------|
| <b>ВВЕДЕНИЕ.....</b>                                                                     | <b>7</b>  |
| <b>1 Теоретические основы и методы ускорения инференса больших языковых моделей.....</b> | <b>12</b> |
| 1.1 Большие языковые модели и задача инференса.....                                      | 12        |
| 1.1.1 Декодер-трансформер и авторегрессионная генерация.....                             | 12        |
| 1.1.2 Последовательное узкое место, латентность и пропускная способность.....            | 13        |
| 1.1.3 Кэш ключей и значений и режим, ограниченный памятью.....                           | 14        |
| 1.2 Спекулятивное декодирование.....                                                     | 15        |
| 1.2.1 Черновая модель, целевая модель и верификатор.....                                 | 15        |
| 1.2.2 Точная верификация через модифицированный rejection sampling .....                 | 16        |
| 1.2.3 Вероятность принятия и формула ускорения.....                                      | 18        |
| 1.3 Ослабленная верификация и компромисс «скорость — качество».....                      | 19        |
| 1.3.1 Нечёткая верификация и порог принятия.....                                         | 19        |
| 1.3.2 Компромисс между пропускной способностью и качеством.....                          | 20        |
| 1.3.3 Ограниченность фиксированных гиперпараметров.....                                  | 21        |
| 1.4 Обзор современных методов спекулятивного декодирования.....                          | 21        |
| 1.4.1 Обучаемая верификация значимых токенов.....                                        | 21        |
| 1.4.2 SpecExec и точная древовидная спекуляция.....                                      | 22        |
| 1.4.3 Облегчённые черновые механизмы: Medusa, EAGLE и родственные.....                   | 22        |
| 1.4.4 Адаптивный выбор длины черновика.....                                              | 23        |
| 1.4.5 Ослабленная адаптивная верификация и фронт Парето.....                             | 24        |
| 1.4.6 Систематизация и выявленный пробел.....                                            | 24        |
| 1.5 Марковские процессы принятия решений как аппарат адаптивного управления.....         | 25        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| 1.5.1    | Определение MDP и уравнение оптимальности Беллмана.....             | 25        |
| 1.5.2    | Метод value iteration: сходимость и вычислительная сложность. .     | 27        |
| 1.5.3    | Моноотонные и пороговые политики.....                               | 27        |
| 1.6      | Исследовательский пробел и постановка задачи.....                   | 28        |
| 1.6.1    | Ограничения существующих адаптивных методов.....                    | 28        |
| 1.6.2    | Идея совместного адаптивного управления.....                        | 29        |
| 1.6.3    | Формальная постановка задачи совместной оптимизации.....            | 29        |
| <b>2</b> | <b>Метод JointAdaSpec и его теоретический анализ.....</b>           | <b>31</b> |
| 2.1      | MDP-формулировка совместной оптимизации.....                        | 31        |
| 2.1.1    | Пространство состояний и его дискретизация.....                     | 31        |
| 2.1.2    | Пространство действий.....                                          | 33        |
| 2.1.3    | Функция переходов и функция награды.....                            | 34        |
| 2.2      | Теоретический анализ метода.....                                    | 36        |
| 2.2.1    | Выборочная сложность оценщика переходов (теорема A).....            | 36        |
| 2.2.2    | Беллман-инвариантность аддитивного штрафа качества (теорема B)..... | 37        |
| 2.2.3    | Субоптимальность каскадных политик (теорема C).....                 | 38        |
| 2.2.4    | Точный value gap совместной и каскадной политик (теорема D).....    | 40        |
| 2.2.5    | Скаляризация Парето-фронта (теорема 2.4).....                       | 41        |
| 2.3      | Алгоритм решения MDP.....                                           | 41        |
| 2.4      | Инференс-алгоритм с выученной политикой.....                        | 43        |
| <b>3</b> | <b>Программная реализация.....</b>                                  | <b>45</b> |
| 3.1      | Архитектура программного комплекса.....                             | 45        |
| 3.2      | Модуль сбора трейсов.....                                           | 47        |
| 3.3      | Модуль оценки MDP и решения value iteration.....                    | 48        |
| 3.4      | Модуль инференса с политикой JointAdaSpec.....                      | 50        |
| 3.5      | Реализация baseline-методов.....                                    | 51        |
| 3.6      | Инструменты, тестирование и воспроизводимость.....                  | 52        |

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>4 Экспериментальное исследование.....</b>                           | <b>55</b> |
| 4.1 Методика экспериментов и метрики.....                              | 55        |
| 4.2 Исследуемые пары моделей и наборы данных.....                      | 56        |
| 4.3 Основной результат: пара 14B/0.5B.....                             | 57        |
| 4.4 Нуль-результат и триангуляция: пара 7B/1.5B.....                   | 58        |
| 4.5 Адаптивность против фиксированного порога (теорема E).....         | 60        |
| 4.6 Анализ компромисса по $k$ и нелинейности качества (теорема G)..... | 61        |
| 4.7 Сравнение с AutoJudge и обсуждение результатов.....                | 62        |
| <b>ЗАКЛЮЧЕНИЕ.....</b>                                                 | <b>65</b> |
| <b>СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....</b>                           | <b>68</b> |
| <b>ПРИЛОЖЕНИЕ А. Доказательства теорем.....</b>                        | <b>71</b> |
| <b>ПРИЛОЖЕНИЕ Б. Архитектура репозитория и листинги.....</b>           | <b>75</b> |
| <b>ПРИЛОЖЕНИЕ В. Дополнительные экспериментальные данные.....</b>      | <b>76</b> |
| <b>ПРИЛОЖЕНИЕ Г. Манифесты воспроизводимости.....</b>                  | <b>78</b> |

## ВВЕДЕНИЕ

Развёртывание больших языковых моделей (large language model, LLM) сместило основную долю вычислительных затрат с обучения на инференс — обслуживание пользовательских запросов. Обучение выполняется однократно, тогда как инференс повторяется при каждом обращении, и по оценкам операторов крупных сервисов на него приходится от 60 до 90 % полной стоимости жизненного цикла модели. Доминирующей операцией здесь служит авторегрессионная генерация: каждый следующий токен вычисляется отдельным прямым проходом по целевой модели и зависит от всех предыдущих. Ответ длиной в несколько сотен токенов требует стольких же последовательных, не распараллеливаемых обращений к модели с десятками миллиардов параметров — именно эта цепочка, а не объём арифметики, определяет наблюдаемую задержку.

Узкое место усугубляется режимом работы памяти. На шаге декодирования обрабатывается ровно один новый токен, но из памяти считываются все веса слоёв и весь накопленный кэш ключей и значений (key-value cache, KV-cache). Арифметическая интенсивность при этом мала, и шаг оказывается ограниченным пропускной способностью памяти (memory-bound), а не производительностью вычислителя; даже на ускорителях класса NVIDIA RTX 5090 с пропускной способностью порядка 1,8 ТБ/с арифметические блоки простаивают в ожидании данных. Отсюда ключевое следствие: проверка нескольких токенов за один проход почти не дороже проверки одного. На этом резерве и построено спекулятивное декодирование.

Спекулятивное декодирование (speculative decoding, SD), предложенное Leviathan и соавторами [10] и независимо Chen и соавторами [4], стало промышленным стандартом ускорения инференса без потери качества и реализовано в системах vLLM, TensorRT-LLM и SGLang. Дешёвая черновая модель за один проход предлагает блок из нескольких токенов, а дорогая целевая модель проверяет их одним параллельным проходом; правило

modified rejection sampling гарантирует, что итоговое распределение принятых токенов совпадает с распределением целевой модели. Выигрыш достигается тогда, когда черновая модель угадывает достаточную долю токенов и за один дорогой проход подтверждается сразу несколько. Однако исходная формулировка использует фиксированную длину черновика и строгое правило приёма, что эмпирически неоптимально.

Литература 2023–2026 годов развивала спекулятивное декодирование по двум во многом независимым направлениям. Первое — адаптивный выбор длины черновика, где число предлагаемых токенов меняется по наблюдаемым признакам генерации: SpecDec++ [8] формулирует остановку черновика как задачу оптимальной остановки с обучаемым классификатором, SVIP и AdaEDL опираются на онлайн-оценку энтропии чернового распределения, DISCO использует контекстные эвристики, BanditSpec — бандитские стратегии. Второе направление — ослабленная (нечёткая) верификация, где условие приёма смягчается ради более высокой пропускной способности: judge decoding и AutoJudge [6] обучают верификатор отбирать значимые расхождения, Fuzzy SD [7] вводит скалярный порог приёма, MARS настраивает ослабление по локальным признакам. Теоретическую границу Парето между скоростью и качеством для ослабленных методов описали Yin и соавторы [18]. Оба направления управляют одной осью при фиксированной другой и не рассматривают их совместно.

Принципиальное ограничение существующих подходов состоит в том, что и длина черновика, и порог верификации, как правило, задаются фиксированными гиперпараметрами, подобранными по усреднённому поведению на валидационной выборке. Между тем оптимальная длина черновика и допустимая степень ослабления проверки зависят от текущего состояния генерации: на «лёгких» участках (типовые окончания, синтаксически предопределённые позиции) выгодны длинный черновик и мягкая проверка, на «трудных» (выбор числа в арифметической задаче, начало



нового шага рассуждения) — короткий черновик и строгая проверка. Любое фиксированное значение оказывается компромиссом, не оптимальным ни для одного конкретного состояния. Более того, последовательное (каскадное) применение двух одномерных адаптивных правил не имеет гарантий оптимальности: оптимальное значение одного параметра функционально зависит от значения другого, поэтому оси нельзя оптимизировать порознь без потери совместного оптимума.

Целью настоящей работы является разработка и экспериментальное исследование метода адаптивного спекулятивного декодирования, который формализует совместное управление длиной черновика и порогом нечёткой верификации как задачу оптимального управления в дискретном марковском процессе принятия решений (Markov decision process, MDP) и решает её методом итерации по ценности (value iteration). Для достижения цели поставлены следующие задачи:

- 1) проанализировать современные методы спекулятивного декодирования, выявить исследовательский пробел, связанный с отсутствием совместной оптимизации двух осей управления, и формализовать задачу совместного управления;

- 2) построить MDP-модель совместной оптимизации — пространства состояний и действий, функции переходов и награды — и доказать её ключевые теоретические свойства;

- 3) реализовать программный комплекс из стадий сбора трейсов, оценки параметров MDP, решения value iteration и инференса с выученной политикой, включая единообразные реализации базовых методов;

- 4) провести воспроизводимую экспериментальную оценку качества и пропускной способности относительно базовых методов на современных открытых моделях семейства Qwen2.5 [13] и установить границы применимости подхода.

Объектом исследования выступает процесс инференса авторегрессионных больших языковых моделей со спекулятивным декодированием. Предметом исследования являются методы совместной адаптивной оптимизации длины черновика и порога верификации, а также их теоретические свойства и эмпирическая эффективность.

Методологическую основу работы составляют теория марковских процессов принятия решений и динамического программирования [12, 14], аппарат приближённого обучения с подкреплением [9] и методы статистической проверки гипотез. Сравнение методов проводится на парных наблюдениях с применением критерия Макнемара и бутстреп-оценки 95 %-х доверительных интервалов; качество измеряется метрикой exact match на наборе математических задач GSM8K [5], скорость — пропускной способностью в токенах в секунду.

Научная новизна работы состоит в том, что длина черновика и порог нечёткой верификации впервые рассматриваются не как независимо настраиваемые гиперпараметры, а как связанные переменные управления в единой MDP-постановке с малым пространством состояний, допускающим точное решение методом value iteration без обучения нейросетевой политики. Для этой постановки получены аналитические результаты: оценка выборочной сложности оценщика переходов, условие Беллман-инвариантности аддитивного штрафа качества, линейная по мере нарушений оценка субоптимальности каскадных политик и точное выражение для разрыва ценности между совместной и каскадной политиками.

Практическая значимость определяется тем, что предложенный метод восстанавливает пропускную способность, теряемую ванильным спекулятивным декодированием на одиночном ускорителе, не требует обучения дополнительных нейросетевых модулей (политика хранится в виде таблицы и стоит  $O(1)$  на шаг декодирования) и совместим с существующими инференс-фреймворками. Реализация оформлена как воспроизводимый

программный комплекс с фиксацией версий, начальных значений генераторов и манифестов. Полученные результаты очерчивают и границы применимости подхода, что важно для инженерных решений о его внедрении.

Работа состоит из введения, четырёх разделов, заключения, списка использованных источников и четырёх приложений. Первый раздел вводит теоретические основы инференса LLM и спекулятивного декодирования, даёт обзор современных методов и аппарата MDP, формулирует исследовательский пробел. Второй раздел излагает метод JointAdaSpec и его теоретический анализ. Третий раздел описывает программную реализацию, четвёртый — экспериментальную оценку. В заключении сформулированы основные результаты и направления дальнейших исследований.

# 1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И МЕТОДЫ УСКОРЕНИЯ ИНФЕРЕНСА БОЛЬШИХ ЯЗЫКОВЫХ МОДЕЛЕЙ

Настоящий раздел вводит понятийный и математический аппарат, на котором строится работа. Сначала рассматривается инференс больших языковых моделей и природа его узкого места (1.1), затем — спекулятивное декодирование как способ обойти это узкое место без потери качества (1.2) и его ослабленные варианты, порождающие компромисс «скорость — качество» (1.3). Раздел 1.4 систематизирует современные методы, раздел 1.5 излагает аппарат марковских процессов принятия решений, а раздел 1.6 формулирует исследовательский пробел и постановку задачи, мотивирующую метод JointAdaSpec.

## 1.1 Большие языковые модели и задача инференса

### 1.1.1 Декодер-трансформер и авторегрессионная генерация

Большая языковая модель (large language model, LLM) — это нейросетевая модель, аппроксимирующая распределение вероятностей над последовательностями токенов естественного языка. Современные LLM семейств LLaMA, Qwen и DeepSeek насчитывают от единиц до сотен миллиардов параметров и почти без исключения построены на архитектуре декодер-трансформера (decoder-only transformer), восходящей к механизму внимания [17]. Модель состоит из стопки одинаковых блоков, каждый из которых сочетает причинно-маскированное самовнимание (causal self-attention) и позиционно-независимую полносвязную подсеть; причинная маска запрещает позиции обращаться к будущим токенам, что и делает модель пригодной для левостороннего (слева направо) порождения текста. Размер словаря  $V$  для перечисленных семейств составляет порядка  $10^5$  токенов.

Пусть  $x$  — входная последовательность (промпт), а  $y = (y_1, \dots, y_n)$  — порождаемый ответ над словарём  $V$ . Декодер-трансформер задаёт

распределение ответа авторегрессионно, то есть как произведение условных распределений каждого токена при условии всех предыдущих:

$$p_{\theta}(y \mid x) = \prod_{t=1}^n p_{\theta}(y_t \mid x, y_{\dot{t}}) \quad (1.1)$$

где

$\theta$  — параметры модели;

$y_{\dot{t}}$  — префикс ранее сгенерированных токенов;

$p_{\theta}(\cdot \mid x, y_{\dot{t}})$  — распределение над словарём  $V$  на шаге  $t$ .

Из выражения (1.1) следует ключевая для всей работы особенность: распределение токена  $y_t$  зависит от уже выбранного значения предыдущего токена. Сгенерировать токен можно лишь после того, как зафиксирован предыдущий, поэтому шаги порождения образуют строго последовательную цепочку. Именно эта зависимость, а не объём арифметики самого по себе, определяет стоимость генерации.

Из условного распределения (1.1) ответ получают одной из стратегий декодирования. Жадное декодирование (greedy) выбирает на каждом шаге наиболее вероятный токен; стохастическое семплирование выбирает токен случайно, часто с температурным масштабированием логитов, регулирующим разнообразие. Для дальнейшего изложения существенно, что спекулятивное декодирование воспроизводит выбранную стратегию целевой модели без искажений, поэтому рассуждение ведётся в терминах целевого распределения  $p$  и переносится на оба режима там, где различие между ними несущественно.

### **1.1.2 Последовательное узкое место, латентность и пропускная способность**

Авторегрессионная генерация ответа длины  $n$  требует  $n$  последовательных прямых проходов (forward pass) по целевой модели. В отличие от стадии обработки промпта (prefill), где все входные токены проходят через модель одним параллельным проходом, стадия декодирования (decode) принципиально последовательна: распараллелить её на уровне

токенов в рамках стандартного подхода невозможно, поскольку вход очередного шага есть выход предыдущего. Это ограничение принято называть последовательным узким местом декодирования (sequential decode bottleneck).

Производительность инференса характеризуют двумя различными метриками. Латентность (latency) — время от запроса до получения ответа; для одиночной последовательности она пропорциональна числу шагов  $n$  и времени одного прохода по целевой модели. Пропускная способность (throughput) — число токенов в секунду (ток/с), генерируемых системой. Эти метрики не эквивалентны: пакетирование (batching) множества запросов повышает суммарную пропускную способность за счёт параллелизма по запросам, но не сокращает число последовательных шагов и потому не улучшает латентность отдельного ответа. Спекулятивное декодирование атакует узкое место с другой стороны — сокращает число обращений к целевой модели на один порождённый токен.

Количественно различие стадий выражается арифметической интенсивностью — отношением числа операций к объёму считанных из памяти данных. На стадии prefill это отношение велико, и стадия ограничена производительностью вычислителя; на стадии decode при batch size, равном единице, оно падает до единиц операций на байт, и стадия ограничена памятью. Пакетирование запросов повышает интенсивность декодирования и потому суммарную пропускную способность, однако, как отмечено выше, не сокращает число последовательных шагов отдельного ответа и не улучшает его латентность.

### **1.1.3 Кэш ключей и значений и режим, ограниченный памятью**

Чтобы на каждом шаге не пересчитывать представления всего префикса заново, используется кэш ключей и значений (key-value cache, KV-cache): векторы ключей и значений механизма внимания, вычисленные для уже обработанных токенов, сохраняются и переиспользуются на последующих шагах. Кэширование снижает арифметическую сложность шага

декодирования с квадратичной до линейной по длине префикса, однако порождает иное ограничение.

На шаге декодирования модель обрабатывает ровно один новый токен, тогда как для вычисления внимания необходимо считать из памяти весь KV-кэш и все веса слоёв. Отношение числа арифметических операций к объёму перемещаемых данных (арифметическая интенсивность) при этом мало, и шаг оказывается ограниченным пропускной способностью памяти (memory-bound), а не производительностью вычислителя (compute-bound). Графический ускоритель в этом режиме недозагружен: его арифметические блоки простаивают в ожидании данных. Практическое следствие двояко. С одной стороны, обработка нескольких токенов за один проход почти не увеличивает время шага по сравнению с обработкой одного — резерв вычислителя позволяет «бесплатно» проверить сразу несколько кандидатов. Именно на этом наблюдении основано спекулятивное декодирование. С другой стороны, рост KV-кэша с длиной контекста увеличивает объём перемещаемых данных и потому остаётся существенным фактором стоимости.

## **1.2 Спекулятивное декодирование**

### **1.2.1 Черновая модель, целевая модель и верификатор**

Спекулятивное декодирование (speculative decoding, SD) ускоряет инференс, вводя в схему вторую, существенно более дешёвую модель [10, 4]. Целевая модель (target model) с распределением  $p$  — это та модель, выборку из которой требуется получить; обычно она велика (единицы–десятки миллиардов параметров) и определяет качество. Черновая модель (draft model) с распределением  $q$  меньше на один–два порядка и служит для дешёвого выдвижения гипотез. Верификатор (verifier) — процедура, которая по предложенным черновиком токенам и распределениям обеих моделей решает, какие из них принять.

Один цикл SD устроен так. Черновая модель авторегрессионно порождает блок из  $\gamma$  токенов-кандидатов ( $\gamma$  называют длиной черновика). Затем целевая модель одним параллельным проходом вычисляет свои распределения  $p$  для всех  $\gamma$  позиций блока сразу — это возможно, поскольку кандидаты уже известны и причинная маска допускает их совместную обработку. Наконец, верификатор сопоставляет  $p$  и  $q$  и принимает максимально длинный согласованный префикс кандидатов, после чего цикл повторяется. Выигрыш возникает оттого, что один дорогой проход целевой модели подтверждает сразу несколько токенов.

Существенным практическим условием применимости SD является совместимость токенизаторов черновой и целевой моделей. Правило приёма сопоставляет вероятности, присвоенные одному и тому же токеноу двумя моделями, поэтому обе обязаны использовать идентичное отображение текста в идентификаторы токенов; иначе сравнение  $p(y_i)$  и  $q(y_i)$  теряет смысл. На практике это ограничивает выбор пар: черновую и целевую модели берут из одного семейства с общим словарём — в настоящей работе из семейства Qwen2.5 [13], где это условие выполнено по построению.

### 1.2.2 Точная верификация через модифицированный rejection sampling

Нетривиальность SD в том, что наивная проверка (принять кандидат, если он совпал с  $\text{argmax}$  целевой модели) искажила бы распределение. Корректность обеспечивает правило модифицированного отклоняющего семплирования (modified rejection sampling) [10]. Кандидат  $y_i$ , выбранный черновиком из  $q$ , принимается с вероятностью

$$a_i = \min\left(1, \frac{p(y_i)}{q(y_i)}\right) \quad (1.2)$$

где

$p(y_i)$  — вероятность токена  $y_i$  по целевой модели;



$q(y_i)$  — вероятность того же токена по черновой модели.

Если кандидат отклонён, на его позиции токен пересемплируется из нормированного остаточного распределения с плотностью, пропорциональной  $(p - q)_+ = \max(0, p - q)$ , а оставшиеся кандидаты блока отбрасываются. Можно показать, что при таком правиле итоговое распределение принятого (или пересемплированного) токена в точности совпадает с распределением целевой модели  $p$ . Тем самым спекулятивное декодирование является не приближённым, а точным методом: оно ускоряет генерацию, не меняя порождаемого распределения. Это утверждение, которое принято называть свойством точности спекулятивного декодирования, доказывается индукцией по числу токенов блока с использованием марковости процесса генерации; полное доказательство вынесено в приложение А. Схема одного цикла приведена на рисунке 1.1.

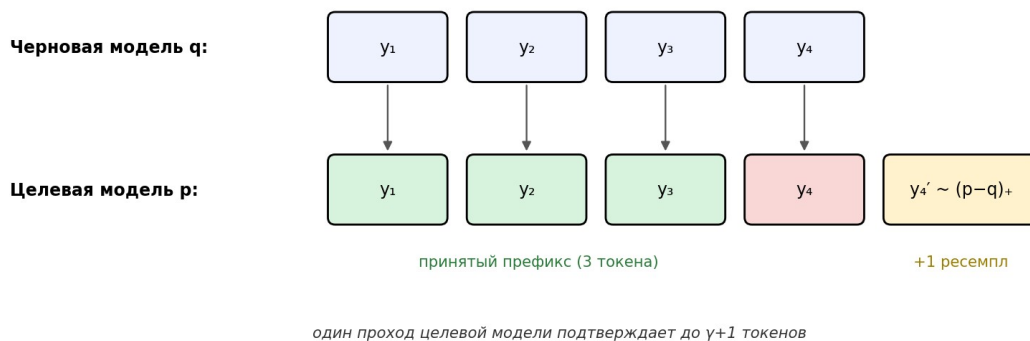


Рисунок 1.1 — Цикл спекулятивного декодирования: черновик предлагает блок токенов, целевая модель проверяет их одним проходом, принятый префикс дополняется одним ресемплом

Принципиально, что выигрыш по скорости достигается без какого-либо переобучения целевой модели и без догадок о «правильности» токенов: правило (1.2) самостоятельно отбраковывает кандидаты, в которых черновик и цель расходятся. Качество ответа в точности соответствует целевой модели, а черновик влияет лишь на скорость.

### 1.2.3 Вероятность принятия и формула ускорения

Ускорение определяется тем, насколько часто кандидаты черновика принимаются. Обозначим через  $\alpha$  ожидаемую вероятность принятия одного токена — меру согласованности черновой и целевой моделей. Если принятия независимы и равновероятны, то ожидаемое число токенов, подтверждаемых за один проход целевой модели при длине черновика  $\gamma$ , равно

$$E[\beta] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha} \quad (1.3)$$

где

$\alpha \in [0, 1]$  — ожидаемая вероятность принятия токена;

$\gamma$  — длина черновика (число кандидатов в блоке);

$\beta$  — число токенов, порождённых за один цикл.

Поскольку каждый цикл стоит одного прохода целевой модели и  $\gamma$  проходов (дешёвой) черновой, итоговый коэффициент ускорения относительно пошаговой генерации равен  $E[\beta] / (1 + \gamma c)$ , где  $c$  — отношение стоимости прохода черновой модели к стоимости прохода целевой. Отсюда видны два источника проигрыша. При низком  $\alpha$  множитель (1.3) близок к единице, и накладные расходы на черновик не окупаются; при малом отношении размеров моделей величина  $c$  велика, и знаменатель растёт, съедая выигрыш от подтверждённых токенов. Оба эффекта наблюдаются эмпирически: на паре Qwen2.5-7B → 1.5B с соотношением мощностей лишь  $4,7\times$  ванильное спекулятивное декодирование оказывается даже медленнее прямой генерации целевой моделью. Это не дефект метода, а прямое следствие формулы (1.3): спекуляция выгодна лишь при достаточно высоком  $\alpha$  и достаточно большом разрыве в стоимости моделей.

Из формулы (1.3) вытекает и теоретический потолок ускорения точного спекулятивного декодирования. При  $\alpha \rightarrow 1$  множитель  $E[\beta]$  стремится к  $\gamma + 1$ , и коэффициент ускорения приближается к  $(\gamma + 1) / (1 + \gamma c)$ , что при больших  $\gamma$  не превосходит  $1/c$  — обратной величины относительной стоимости чернового

прохода. Преодолеть этот потолок, оставаясь в классе точных методов, невозможно: дальнейший выигрыш достигим либо повышением согласованности  $\alpha$  за счёт более удачной черновой модели, либо контролируемым ослаблением правила приёма, которое рассматривается в разделе 1.3.

Эмпирически относительная стоимость  $s$  прямо определяет применимость метода. На основной паре (целевая модель 14В, черновая 0,5В) отношение мощностей около 28-кратного, величина  $s$  мала, и потолок  $1/s$  высок; на дополнительной паре (7В и 1,5В) отношение лишь 4,7-кратное,  $s$  велико, и потолок низок. Этим и объясняется наблюдаемое в разделе 4 явление: на второй паре ванильное спекулятивное декодирование оказывается медленнее прямой генерации, поскольку дешёвого черновика, способного окупить накладные расходы, при таком отношении попросту нет.

### **1.3 Ослабленная верификация и компромисс «скорость — качество»**

#### **1.3.1 Нечёткая верификация и порог принятия**

Точное правило (1.2) консервативно: оно отклоняет кандидат всякий раз, когда целевая модель присваивает ему меньшую вероятность, чем черновая. Однако на практике многие такие токены семантически приемлемы, и их отклонение лишь снижает скорость. Нечёткая (ослабленная) верификация (fuzzy verification) смягчает условие принятия, вводя порог верификации (verification threshold)  $T \geq 1$ : кандидат принимается, если

$$\frac{p(y_i)}{q(y_i)} \geq \frac{1}{T} \quad (1.4)$$

где

$T \geq 1$  — порог верификации (множитель допуска);

при  $T = 1$  правило (1.4) совпадает с точным условием, и SD остаётся точным;

при  $T > 1$  допускается принятие токенов, недооценённых целевой моделью.

Тем самым порог  $T$  непрерывно интерполирует между точным спекулятивным декодированием ( $T = 1$ ) и всё более агрессивным приёмом черновых токенов ( $T \rightarrow \infty$ ). При  $T > 1$  метод становится приближённым (lossy): распределение порождаемых токенов отклоняется от распределения целевой модели. Именно в такой скалярной форме порог приёма введён в методе Fuzzy SD [7]; его удобство для настоящей работы в том, что единственная вещественная величина  $T$  полностью параметризует ось верификации и потому естественно включается в пространство действий управляющего процесса (раздел 1.5 и далее).

### **1.3.2 Компромисс между пропускной способностью и качеством**

Порог  $T$  задаёт прямой компромисс между пропускной способностью и качеством. Чем больше  $T$ , тем выше доля принятых токенов  $\alpha$ , а значит, согласно (1.3), и число токенов за цикл — пропускная способность растёт. Одновременно растёт и доля токенов, в которых система доверилась черновой модели вопреки целевой, — качество ответа отклоняется от эталонного. На «лёгких» участках последовательности ( типовые окончания, синтаксически предопределённые позиции) ослабление проверки почти безвредно; на «трудных» (выбор числа в арифметической задаче, начало нового шага рассуждения) то же ослабление может изменить смысл ответа. Качество, таким образом, нелинейно зависит от доли принятий, и эта зависимость неоднородна по ходу генерации.

Множество достижимых пар «пропускная способность — качество» при изменении  $T$  образует границу Парето: ни одну из двух величин нельзя улучшить, не ухудшив другую. Теоретическую структуру этой границы для семейств методов с параметризованным ослаблением верификации исследовали Yin и соавторы [18], показавшие, что при разумных предположениях фронт монотонен. Настоящая работа опирается на этот

результат дважды: как на язык описания компромисса (раздел 4) и как на ориентир для скаляризации двухкритериальной задачи управления (раздел 2).

### **1.3.3 Ограниченность фиксированных гиперпараметров**

И длина черновика  $\gamma$ , и порог  $T$  в большинстве реализаций задаются фиксированными значениями, подобранными по усреднённому поведению на валидационной выборке. Между тем из изложенного следует, что оптимальные  $\gamma$  и  $T$  зависят от текущего состояния генерации. Когда черновая модель уверена и согласована с целевой, выгодны длинный черновик и мягкий порог; когда уверенность мала, разумнее короткий черновик и строгая проверка, чтобы не тратить проходы впустую и не портить качество. Любое фиксированное значение есть компромисс, не оптимальный ни для одного конкретного состояния. Это наблюдение и мотивирует переход к адаптивному управлению, в котором  $\gamma$  и  $T$  выбираются как функции состояния.

## **1.4 Обзор современных методов спекулятивного декодирования**

Развитие спекулятивного декодирования в 2023–2026 годах удобно разложить по трём направлениям: совершенствование правила приёма, конструирование более точной черновой модели и адаптивное управление гиперпараметрами цикла. Ниже методы рассмотрены в этом порядке, после чего сведены в таблицу 1.1 по двум осям управления, существенным для настоящей работы, — длине черновика и критерию верификации.

### **1.4.1 Обучаемая верификация значимых токенов**

Первое направление заменяет вероятностное правило (1.2) обучаемым критерием, оценивающим не числовое расхождение моделей, а его значимость для итогового ответа. В judge decoding роль верификатора играет классификатор, принимающий черновой токен, если тот семантически допустим, даже при низком отношении  $p / q$ . Метод AutoJudge [6] доводит эту идею до автоматического обучения: метки добываются без ручной разметки по эквивалентности ответов на наборе GSM8K [5] — токен считается

незначимым, если его замена не меняет правильность решения, — после чего на размеченных примерах обучается логистическая регрессия с калибровкой на целевую полноту. На инференсе классификатор пропускает «незначимые» расхождения и повышает долю принятий; в собственных измерениях (раздел 4) на паре 7B/1.5B AutoJudge достигает 61,7 % exact match. Родственный SelfJudge устраняет отдельную фазу обучения, используя в роли judge саму черновую модель ценой дополнительных проходов. Общее свойство группы — высокий эффективный acceptance rate при приближённом (lossy) характере генерации и, как правило, потребности в обучении отдельного верификатора под каждую задачу.

#### **1.4.2 SpecExes и точная древовидная спекуляция**

Метод SpecExes [15] увеличивает число подтверждаемых за цикл токенов, переходя от линейного черновика к дереву кандидатов. Черновая модель строит префиксное дерево наиболее вероятных продолжений, целевая модель проверяет все ветви одним проходом с переиспользованием KV-кэша вдоль рёбер дерева, после чего выбирается наилучший допустимый путь. За счёт пакетного внимания верификация дерева стоит примерно столько же, сколько верификация линейной цепочки той же глубины, тогда как ожидаемое число принятых токенов выше. SpecExes сохраняет точное распределение целевой модели и особенно эффективен при больших бюджетах параллелизма, однако управляет структурой дерева, а не порогом приёма, и потому ортогонален оси верификации.

#### **1.4.3 Облегчённые черновые механизмы: Medusa, EAGLE и родственные**

Третья группа улучшает не правило проверки, а сам механизм выдвижения гипотез, повышая согласованность  $\alpha$  при низкой стоимости  $s$ . В методе Medusa [3] к целевой модели пристраивается несколько дополнительных голов (decoding heads), параллельно предсказывающих токены на несколько позиций вперёд; их кандидаты объединяются

древовидным вниманием. EAGLE [11] выполняет авторегрессию не на уровне токенов, а на уровне скрытых представлений (hidden states) целевой модели, что заметно уменьшает дрейф распределений между черновиком и целью. К этому же направлению относятся Kangaroo, где черновиком служит подмножество слоёв целевой модели с лёгким адаптером, и LayerSkip, реализующий self-speculative decoding через ранний выход из промежуточных слоёв. Все они повышают  $\alpha$ , но требуют специального обучения и не вводят управляемого порога качества. По отношению к настоящей работе эти методы ортогональны: предложенное управление парой (длина, порог) применимо поверх любой из таких черновых моделей.

Перечисленные подходы различаются балансом между ростом согласованности  $\alpha$  и ростом относительной стоимости  $c$ . Medusa почти не увеличивает  $c$  (головы дешёвы), но прирост  $\alpha$  ограничен независимостью голов; EAGLE на уровне скрытых представлений достигает большего  $\alpha$  ценой обучения авторегрессионной черновой сети, а версия EAGLE-3 дополнительно улучшает обучение и архитектуру по сравнению с ранними EAGLE-1 и EAGLE-2. Kangaroo и LayerSkip переиспользуют слои самой целевой модели, снижая  $c$  за счёт более тесной связи с целью. Для настоящей работы существенно, что любой из этих механизмов лишь смещает рабочую точку ( $\alpha$ ,  $c$ ), не затрагивая управление парой (длина, порог), и потому сочетается с предлагаемым методом, а не конкурирует с ним.

#### **1.4.4 Адаптивный выбор длины черновика**

Ближайшее к настоящей работе направление управляет длиной черновика  $\gamma$ , сохраняя строгое правило приёма. В методе SpecDec++ [8] решение «продолжать ли черновик» формулируется как задача оптимальной остановки: обучаемый классификатор предсказывает вероятность принятия следующего токена по скрытым представлениям черновой модели, и генерация прерывается при падении этой вероятности ниже порога. Методы SVIP и AdaEDL опираются на онлайн-оценку энтропии чернового

распределения — высокая энтропия служит предиктором низкой вероятности принятия; по существу оба реализуют одномерный пороговый критерий в координатах энтропии. DISCO выбирает длину по контекстным эвристикам без обучаемой политики, а BanditSpec трактует выбор  $\gamma$  как задачу о многоруком бандите с наградой в виде числа принятых токенов за единицу времени. Все эти методы адаптируют ровно одну ось — длину  $\gamma$  — при фиксированном (как правило, точном) правиле проверки, и потому их выигрыш ограничен сверху потолком  $1/c$  из подраздела 1.2.3.

#### **1.4.5 Ослабленная адаптивная верификация и фронт Парето**

Зеркально к предыдущей группе действуют методы, фиксирующие длину черновика и управляющие осью верификации. Fuzzy SD [7] вводит скалярный порог приёма  $T$  (подраздел 1.3.1), переводящий метод в lossy-режим; MARS обобщает эту идею через matched acceptance rule, настраивая степень ослабления по локальным признакам распределения целевой модели. MARS — наиболее близкий к настоящей работе метод по оси верификации; принципиальное отличие предлагаемого подхода в том, что порог  $T$  оптимизируется не изолированно, а совместно с длиной  $\gamma$ . Теоретическую рамку для всей группы задаёт результат Yin и соавторов [18] о структуре границы Парето между ускорением и расхождением распределений: при разумных предположениях фронт монотонен, что и обосновывает скаляризацию двухкритериальной задачи через множитель Лагранжа в разделе 2.

#### **1.4.6 Систематизация и выявленный пробел**

Рассмотренные методы удобно классифицировать по двум независимым осям управления — характеру выбора длины черновика (фиксированная, эвристическая, обучаемая) и строгости верификации (строгая, ослабленная фиксированная, ослабленная адаптивная). Соответствующая систематизация приведена в таблице 1.1.



Таблица 1.1 — Систематизация методов спекулятивного декодирования по осям управления длиной черновика и критерием верификации

| Длина \<br>Верификация    | Строгая<br>(MRS)                        | Ослабленная<br>фиксированная                           | Ослабленная<br>адаптивная   |
|---------------------------|-----------------------------------------|--------------------------------------------------------|-----------------------------|
| Фиксированная             | SD [10, 4]                              | Judge Decoding, AutoJudge [6], SelfJudge, Fuzzy SD [7] | MARS                        |
| Эвристическая             | DISCO                                   | —                                                      | —                           |
| Обучаемая /<br>адаптивная | SpecDec++ [8], SVIP, AdaEDL, BanditSpec | —                                                      | JointAdaSpec (наст. работа) |

Таблица 1.1 делает пробел явным. Ячейка на пересечении адаптивного управления длиной и ослабленной адаптивной верификации остаётся незаполненной: ни одна из рассмотренных работ не управляет обеими осями совместно как связанными переменными единой оптимизационной задачи. Частные комбинации встречаются (например, адаптивную длину применяют поверх фиксированного порога Fuzzy SD), однако такое каскадное сочетание не обеспечивает совместного оптимума, поскольку оптимальное значение одного параметра функционально зависит от другого. Именно эту нишу занимает предлагаемый метод JointAdaSpec, причём, в отличие от AutoJudge, Medusa и EAGLE, он не требует обучения нейросетевых модулей — управляющая политика хранится в виде таблицы.

## 1.5 Марковские процессы принятия решений как аппарат адаптивного управления

### 1.5.1 Определение MDP и уравнение оптимальности Беллмана

Естественным формализмом для последовательного выбора действий под неопределённостью служит марковский процесс принятия решений (Markov decision process, MDP) [12]. MDP задаётся кортежем  $(S, A, P, r, \gamma)$ , где  $S$  — множество состояний,  $A$  — множество действий,  $P(s' | s, a)$  — вероятность перехода в  $s'$  при выборе  $a$  в  $s$ ,  $r(s, a)$  — мгновенная награда, а  $\gamma \in [0, 1)$  — коэффициент дисконтирования, придающий больший вес близким во времени наградам. Политика  $\pi$  сопоставляет состоянию действие; её ценность есть

математическое ожидание дисконтированной суммы наград при следовании  $\pi$  из состояния  $s$ . Оптимальная политика максимизирует ценность во всех состояниях, а её функция ценности удовлетворяет уравнению оптимальности Беллмана:

$$V^*(s) = \max_{a \in A} \left[ r(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s') \right] \quad (1.5)$$

где

$V^*(s)$  — оптимальная ценность состояния  $s$ ;

$r(s, a)$  — награда за действие  $a$  в состоянии  $s$ ;

$\gamma$  — коэффициент дисконтирования;

$P(s' | s, a)$  — вероятность перехода в состояние  $s'$ .

Удобно ввести функцию ценности действия (Q-функцию): величина  $Q^*(s, a)$  складывается из мгновенной награды  $r(s, a)$  и дисконтированной ожидаемой ценности следующего состояния. Оптимальное действие в состоянии  $s$  доставляет максимум  $Q^*(s, a)$  по действиям  $a$ ; тем самым, зная  $V^*$ , политику извлекают жадно. Применительно к спекулятивному декодированию состояние описывает текущую ситуацию генерации, а действие — выбор длины черновика и порога, что и превращает задачу адаптивного управления в задачу решения MDP. Оговорим коллизию обозначений: символ  $\gamma$  традиционно используют и для длины черновика (разделы 1.2–1.4), и для коэффициента дисконтирования (настоящий раздел). Во избежание неоднозначности при построении MDP-формулировки JointAdaSpec в разделе 2 коэффициент дисконтирования переобозначается через  $\lambda$ , а  $\gamma$  сохраняется за длиной черновика.

Качество MDP-модели целиком определяется выбором состояния: оно должно быть достаточно информативным, чтобы оптимальное действие зависело лишь от него (свойство марковости), и достаточно компактным, чтобы пространство  $S$  оставалось малым и допускало точное решение. В разделе 2 состояние составляется из трёх наблюдаемых на каждом цикле

признаков — энтропии черного распределения как меры уверенности черновика, накопленного расхождения Кульбака — Лейблера между черновым и целевым распределениями как меры их рассогласования и позиции внутри текущего блока, — и каждый из них дискретизируется в конечное число интервалов. Такой выбор делает состояние наблюдаемым в ходе обычного цикла SD без дополнительных вычислений.

### 1.5.2 Метод value iteration: сходимость и вычислительная сложность

Стандартный способ найти  $V^*$  — итерация по ценности (value iteration), представляющая собой последовательное применение оператора Беллмана:

$$V_{k+1}(s) = \max_{a \in A} \left[ r(s, a) + \gamma \sum_{s'} P(s' | s, a) V_k(s') \right] \quad (1.6)$$

где

$V_k$  — приближение функции ценности на итерации  $k$ .

Оператор Беллмана является сжатием с коэффициентом  $\gamma$  в равномерной норме, поэтому итерации (1.6) сходятся к единственной неподвижной точке  $V^*$  геометрически — с линейной скоростью, определяемой коэффициентом сжатия  $\gamma$  [12]. Метод восходит к динамическому программированию [2]; одна итерация требует  $O(|S| \cdot |A|)$  операций при разреженной функции переходов, а число итераций до достижения точности  $\epsilon$  составляет порядка  $\log(1/\epsilon) / (1 - \gamma)$ . При конечных и невысоких по размерности  $S$  и  $A$  он даёт точное решение без аппроксимации функции ценности, что выгодно отличает его от градиентных методов обучения с подкреплением [14]. Качество табличного решения, однако, ограничено точностью оценки  $P$  и  $r$  по конечной выборке: оценки выборочной сложности для подобных оценщиков, построенных по выборке из генеративной модели, известны [1] и используются во втором разделе при обосновании теоремы о точности оценщика переходов.

### 1.5.3 Монотонные и пороговые политики

Для многих прикладных MDP оптимальная политика имеет регулярную структуру: если состояния и действия упорядочены, а награда и переходы

удовлетворяют условиям монотонности и супермодулярности [16], то оптимальное действие монотонно по состоянию, а сама политика приобретает пороговый вид — действие меняется при пересечении некоторой границы в пространстве состояний [12]. Пороговые политики удобны и вычислительно, и интерпретационно; именно к ним сводятся одномерные адаптивные методы из подраздела 1.4.4: SVIP и AdaEDL реализуют пороговое правило в координате энтропии черного распределения, SpecDec++ — в координате обучаемой оценки вероятности принятия. Настоящая работа обобщает эту одномерную пороговую структуру на двумерное управление парой (длина, порог). Вместе с тем выполнение условий супермодулярности не гарантировано: при их нарушении пороговая — а значит, и каскадная, рассматривающая оси по очереди — структура перестаёт быть оптимальной. Проверка этих условий и количественная оценка последствий их нарушения составляют предмет теоретического анализа во втором разделе.

## **1.6 Исследовательский пробел и постановка задачи**

### **1.6.1 Ограничения существующих адаптивных методов**

Проведённый обзор позволяет точно сформулировать пробел. Методы адаптивной длины (SpecDec++ и родственные) управляют осью  $\gamma$  при фиксированном правиле проверки; методы ослабленной верификации (нечёткое SD, AutoJudge) управляют осью качества при фиксированной или эвристической длине. Ни одна из работ не рассматривает  $\gamma$  и  $T$  как связанные переменные единой оптимизационной задачи. Между тем оси взаимозависимы: выгодная длина черновика зависит от того, насколько строг порог приёма, и наоборот. Последовательное (каскадное) применение двух одномерных правил — выбрать сначала длину, затем порог (или наоборот) — кажется естественным, но не имеет гарантий оптимальности именно из-за этой взаимозависимости.

### **1.6.2 Идея совместного адаптивного управления**

Предлагаемый подход состоит в том, чтобы трактовать выбор пары (длина черновика, порог верификации) на каждом цикле декодирования как действие в марковском процессе принятия решений, состояние которого описывает текущую ситуацию генерации (уверенность черновика, накопленное расхождение моделей, позицию в блоке). Оптимальная политика такого MDP по построению учитывает связь осей: она назначает совместно оптимальную пару  $(\gamma, T)$  для каждого состояния. Каскадные эвристики при этом оказываются частным случаем — подмножеством совместных политик, — что позволяет строго сравнить их с совместным решением. Такой взгляд переводит инженерную задачу подбора гиперпараметров в задачу оптимального управления с доказуемыми свойствами.

### **1.6.3 Формальная постановка задачи совместной оптимизации**

Формально требуется построить дискретный MDP  $(S, A, P, r, \gamma)$ , в котором состояние агрегирует наблюдаемые признаки цикла спекулятивного декодирования, действие задаёт пару (длина черновика, уровень порога верификации), награда поощряет число принятых токенов и штрафует затраченное время и потерю качества, а решение уравнения (1.5) методом (1.6) даёт совместно оптимальную управляющую политику. От искомого решения требуется: сохранять принцип спекулятивного декодирования (управляемое отклонение от целевого распределения через порог  $T$ ); допускать точное и воспроизводимое вычисление политики без обучения нейросетевых модулей; и поддаваться теоретическому анализу — в части корректности оценки параметров, влияния штрафа качества и соотношения совместной и каскадной политик. Построению такого MDP, его решению и теоретическому анализу посвящён второй раздел работы.

Двухкритериальность задачи — одновременная максимизация пропускной способности и качества — разрешается скаляризацией: потеря качества входит в награду как аддитивный штраф с неотрицательным

множителем  $\kappa$ . Изменение  $\kappa$  перемещает оптимум вдоль границы Парето из подраздела 1.3.2: при  $\kappa = 0$  максимизируется только скорость, при росте  $\kappa$  растёт вес качества. Тем самым единая постановка охватывает целое семейство компромиссов, а не одну фиксированную точку, что отличает её от методов с заранее заданным порогом.

## 2 МЕТОД JOINTADASPEC И ЕГО ТЕОРЕТИЧЕСКИЙ АНАЛИЗ

Первый раздел установил исследовательский пробел: ни один из существующих методов не управляет длиной черновика и порогом верификации совместно как связанными переменными единой оптимизационной задачи. Настоящий раздел закрывает этот пробел. Сначала строится дискретный марковский процесс принятия решений, состояние которого агрегирует наблюдаемые признаки цикла спекулятивного декодирования, а действие задаёт пару (длина, порог) (2.1); затем проводится теоретический анализ свойств оптимальной политики (2.2); наконец, описываются алгоритм решения MDP методом итерации по ценности (2.3) и алгоритм инференса с выученной политикой (2.4). Во всём разделе коэффициент дисконтирования обозначается через  $\lambda$ , а символ  $\gamma$  сохранён за длиной черновика (см. замечание об обозначениях в подразделе 1.5.1).

### 2.1 MDP-формулировка совместной оптимизации

#### 2.1.1 Пространство состояний и его дискретизация

Состояние MDP должно быть достаточно информативным, чтобы оптимальная пара (длина, порог) зависела лишь от него (свойство марковости), и достаточно компактным для точного табличного решения. Эти требования удовлетворяет тройка наблюдаемых на каждом шаге величин: энтропия чернового распределения, расхождение чернового и целевого распределений и позиция в текущем блоке.

Энтропия чернового распределения  $H$ . Для текущей позиции черновика энтропия Шеннона распределения  $q$  вычисляется как

$$H(q_i) = - \sum_{x \in V} q_i(x) \log q_i(x) \quad (2.1)$$

где

$q_i$  — распределение черновой модели на позиции  $i$ ;

$V$  — словарь токенов.

Энтропия  $H$  служит индикатором локальной неуверенности черновой модели: при низкой энтропии черновик уверен в следующем токене, что повышает шансы согласия с целевой моделью и принятия токена. Именно этот признак лежит в основе одномерных методов SVIP и AdaEDL (подраздел 1.4.4); его включение сохраняет содержательную преемственность с литературой. В работе используется верхняя граница  $H_{\max} = 6,0$  нат, покрывающая свыше 99 % наблюдаемых на трейсах значений.

Дивергенция Кульбака — Лейблера  $K$ . Расхождение между черновым и целевым распределениями измеряется дивергенцией

$$K(q_i, p_i) = \text{KL}(q_i \parallel p_i) = \sum_{x \in V} q_i(x) \log \frac{q_i(x)}{p_i(x)} \quad (2.2)$$

Если энтропия  $H$  характеризует абсолютную неуверенность черновой модели, то  $K$  характеризует именно расхождение распределений. Высокое  $K$  при низком  $H$  отвечает ситуации «черновик уверен, но неверно»: точное правило (1.2) отвергнет такой токен с высокой вероятностью. Совместное наблюдение пары  $(H, K)$  несёт существенно больше информации о предстоящем принятии, чем любая одномерная статистика. Использовано направление  $\text{KL}(q \parallel p)$ , а не обратное: кандидаты выбираются из  $q$ , поэтому расхождение « $q$  относительно  $p$ » точнее предсказывает поведение верификации. Верхняя граница  $K_{\max} = 8,0$  нат.

Совместное наблюдение пары  $(H, K)$  различает четыре качественно разных режима. Низкие  $H$  и  $K$  отвечают согласию уверенных моделей — выгоден длинный черновик с мягким порогом. Высокое  $H$  при низком  $K$  — обе модели неуверены, но их распределения близки. Низкое  $H$  при высоком  $K$  — черновик уверен, но расходится с целью: типичная ловушка, в которой ослаблять проверку опасно. Высокие  $H$  и  $K$  — общая неопределённость, при которой осторожнее короткий черновик. Ни одна одномерная статистика эти режимы не разделяет, что и обосновывает двумерное состояние.



Позиция в черновике  $k$ . Дискретный счётчик уже сгенерированных в текущем блоке токенов необходим по двум содержательным причинам. Маргинальная выгода от продолжения черновика падает с ростом  $k$  в силу геометрического убывания вероятности достичь позиции  $k + 1$ , а верхняя граница  $\gamma_{\max}$  создаёт жёсткое ограничение, которое политика обязана учитывать, чтобы не вырождаться в правило «продолжать всегда». Без  $k$  процесс перестаёт быть марковским. Принято  $\gamma_{\max} = 8$ .

Итоговое пространство состояний есть произведение

$$S = [0, H_{\max}] \times [0, K_{\max}] \times [0, 1, \dots, \gamma_{\max}] \quad (2.3)$$

Для табличного решения непрерывные оси  $H$  и  $K$  дискретизируются равномерной сеткой  $20 \times 20$ , что вместе с девятью значениями  $k \in \{0, \dots, 8\}$  даёт  $|S| = 20 \cdot 20 \cdot 9 = 3600$  состояний. Альтернативные признаки отвергнуты по содержательным причинам:  $\text{top-1}$  вероятность и  $\text{margin}$  черновика сильно коллинеарны энтропии  $H$ ; скрытое представление черновой модели имеет размерность в тысячи и исключает табличный MDP; глобальный счётчик принятых токенов с начала генерации не является марковским.

### 2.1.2 Пространство действий

Действие на каждом шаге двумерно и отражает два независимых рычага управления — продолжать ли генерацию черновика и какой порог применить при верификации следующего токена. Действие по оси длины бинарно:

$$a_{\text{length}} \in A_{\text{length}} = \{\text{stop}, \text{continue}\} \quad (2.4)$$

Действие  $\text{stop}$  останавливает генерацию черновика и переводит цикл к фазе верификации;  $\text{continue}$  порождает следующий черновой токен. При  $k = \gamma_{\max}$  действие принудительно равно  $\text{stop}$ . Действие по оси верификации — скалярный порог  $T$  из семейства нечёткой верификации (1.4); непрерывный диапазон  $[1, T_{\max}]$  дискретизируется по геометрической сетке

$$a_{\text{verif}} \in A_{\text{verif}} = \{1, T_{\max}^{1/M}, T_{\max}^{2/M}, \dots, T_{\max}\} \quad (2.5)$$

Геометрическая сетка выбрана потому, что влияние порога на вероятность принятия мультипликативно: переход от  $T = 1$  к  $T = 1,1$  и переход от  $T = 5$  к  $T = 5,5$  содержательно различны. При  $T_{\max} = 4,0$  и  $M = 7$  получаются восемь уровней  $\{1,00; 1,22; 1,49; 1,82; 2,22; 2,71; 3,30; 4,00\}$ ; ограничение  $T_{\max} = 4$  обусловлено тем, что при больших порогах распределение порождаемых токенов отклоняется от целевого настолько, что деградация качества становится неприемлемой. Совместное пространство действий есть произведение осей

$$A = A_{length} \times A_{verif}, |A| = 2 \cdot 8 = 16 \quad (2.6)$$

Размер  $|A| = 16$  достаточен для разрешения совместной политики при стоимости одной итерации решения порядка  $O(|S| \cdot |A|)$ , что составляет несколько секунд на одном процессоре.

### 2.1.3 Функция переходов и функция награды

Функция переходов  $P(s' \mid s, a)$  не допускает аналитической формы: признаки нового состояния  $(H', K')$  функционально определяются распределениями  $q$  и  $r$ , которые сами суть результат прямого прохода через многослойные сети. Поэтому вместо аналитической формы используется эмпирическая оценка, получаемая из трейсов спекулятивного декодирования (процедура описана в подразделе 2.3). Содержательно различимы переходы после `continue` (порождается следующий черновой токен ценой одного вызова черновой модели) и после `stop` (выполняется верификация ценой одного вызова целевой модели и начинается новый блок).

Формально различимы четыре типа переходов: продолжение черновика (`continue` при  $k < \gamma_{\max}$ ), его остановка (`stop`), принудительная остановка на границе (`continue` при  $k = \gamma_{\max}$  сводится к `stop`) и переход к началу нового блока после верификации. Существенное для теоремы 2.3 условие стохастической монотонности (C2) содержательно означает, что сдвиг текущего состояния к большим  $(H, K)$  делает более вероятным и большее  $(H', K')$  следующего состояния. Эта монотонность отражает локальную

автокоррелированность сложности текста: высокая неуверенность черновика в текущей позиции свидетельствует о локально сложном участке последовательности. Эмпирическая проверка условий монотонности вынесена в подраздел 4.х.

Функция награды выводится из операционной цели метода — максимума числа принятых токенов в единицу времени при ограничении на качество. Обозначив через  $\eta$  пропускную способность (принятых токенов в секунду), а через  $D$  — расхождение распределения порождаемых токенов с распределением целевой модели, содержательную задачу записывают как условную оптимизацию

$$\eta(\pi) \rightarrow \max, \quad D(\pi) \leq \varepsilon \quad (2.7)$$

где  $\varepsilon$  — допустимый уровень потери качества. Применяя метод множителей Лагранжа к ограничению (2.7), переходят к безусловной задаче с пошаговой наградой

$$r(s, a) = \eta(s, a) - \kappa \cdot D(s, a) \quad (2.8)$$

в которой множитель  $\kappa \geq 0$  задаёт цену единицы потери качества и тем самым выбирает точку на границе Парето из подраздела 1.3.2: при  $\kappa = 0$  максимизируется чистая скорость, с ростом  $\kappa$  растёт вес качества. Оптимальная политика есть решение уравнения оптимальности Беллмана с этим вознаграждением

$$V^i(s) = \max_{a \in A} \left[ r(s, a) + \lambda \sum_{s'} P(s' | s, a) V^i(s') \right] \quad (2.9)$$

Существенно, что штраф качества входит в награду через слагаемое, зависящее от действия (выбранного порога  $T$ ): как показывает теорема В (подраздел 2.2.2), штраф, зависящий только от состояния, не влияет на оптимальную политику. Именно связанность награды с действием делает управление качеством содержательным.

Конкретная форма награды в реализации (подраздел 3.3) такова: число принятых токенов поощряется, затраченное время штрафуются небольшим множителем  $s\_time = 0,01$ , а потеря качества — слагаемым с множителем  $k$ . Аддитивный штраф качества, согласующийся с теоремой В, параметризуется коэффициентами при дивергенции  $K$  и позиции  $k$  (поля  $quality\_risk\_K$  и  $quality\_risk\_k$  конфигурации); по умолчанию оба равны нулю, что соответствует базовой постановке (2.8). Малое  $s\_time$  гарантирует, что доминирующим членом награды остаётся число принятых токенов, а штрафы лишь корректируют выбор на «трудных» состояниях.

## 2.2 Теоретический анализ метода

Теоретическая часть отвечает на три вопроса: насколько точно политику можно выучить из конечной выборки трейсов (2.2.1), как корректно ввести штраф качества (2.2.2) и в каком отношении находятся совместная и каскадная политики (2.2.3–2.2.5). Формулировки теорем приводятся с указанием идеи доказательства; полные выкладки вынесены в приложение А.

### 2.2.1 Выборочная сложность оценщика переходов (теорема А)

Табличная политика вычисляется по эмпирической оценке функции переходов, поэтому качество решения ограничено точностью этой оценки на конечной выборке. Следующая теорема даёт равномерную по состояниям оценку отклонения выученной функции ценности от истинной.

Теорема А (выборочная сложность). Пусть  $\hat{P}$  — оценка функции переходов по выборке, в которой каждая пара (состояние, действие) наблюдалась не менее  $n_{min}$  раз, и пусть применяется сглаживание Лапласа с параметром  $\alpha$ . Тогда с вероятностью не менее  $1 - \delta$

$$\|\hat{V} - V^i\|_{\infty} \leq \frac{2R}{(1 - \lambda)^2} \sqrt{\frac{2 \ln \left( \frac{2|S||A|}{\delta} \right)}{n_{min}}} + \frac{\alpha |S| R}{(1 - \lambda)(n_{min} + \alpha)} \quad (2.10)$$

где

$R$  — верхняя граница модуля награды;

$\lambda$  — коэффициент дисконтирования;

$\delta$  — уровень доверия.

Первое слагаемое — статистическая ошибка (концентрация Хёфдинга по выборке переходов), убывающая как корень из числа наблюдений; второе — смещение, вносимое сглаживанием Лапласа и убывающее с ростом  $n_{\min}$ . Практический вывод: для контроля ошибки достаточно гарантировать минимальную заполненность каждой пары (состояние, действие) в трейсах, что и закладывается в процедуру сбора (2.3).

Идея доказательства. Для фиксированной пары  $(s, a)$  оценка функции переходов есть нормированный счётчик наблюдений, и по неравенству Хёфдинга её отклонение от истинного распределения убывает как корень из числа наблюдений. Объединение оценок по всем  $|S| \cdot |A|$  парам (union bound) даёт логарифмический множитель  $\ln(2|S| \cdot |A| / \delta)$ . Перенос ошибки оценки переходов на ошибку функции ценности опирается на  $\lambda$ -сжимаемость оператора Беллмана и вносит множитель  $1 / (1 - \lambda)^2$ : одна степень отвечает за горизонт планирования, вторая — за накопление ошибки вдоль итераций. Смещение от сглаживания Лапласа учитывается отдельным слагаемым, линейным по  $\alpha$  и убывающим с ростом  $n_{\min}$ . Полные выкладки приведены в приложении А.

### **2.2.2 Беллман-инвариантность аддитивного штрафа качества (теорема В)**

Конструкция награды (2.8) вводит штраф качества как слагаемое, зависящее от действия. Следующее утверждение объясняет, почему это необходимо, и одновременно корректирует более раннюю мультипликативную форму штрафа, нарушавшую сжимаемость оператора Беллмана.

Теорема В (Беллман-инвариантность). Пусть к награде добавлен штраф вида  $g(s)$ , зависящий только от состояния. Тогда оптимальная политика MDP не изменяется: добавка сдвигает функцию ценности на постоянную по действиям

величину и сокращается в операторе максимума. Форму оптимальной политики способен изменить лишь штраф, связанный с действием,  $g(s, a)$ .

Содержательное следствие двояко. Прежде всего, штраф качества обязан зависеть от выбранного порога  $T$  — иначе, по доказанному, он не влияет на политику; это и заложено в (2.8). Кроме того, аддитивная форма штрафа сохраняет  $\lambda$ -сжимаемость оператора Беллмана, а с ней и гарантии сходимости итерации по ценности (теорема 1.4), тогда как мультипликативная форма их разрушала. Именно поэтому в окончательной формулировке метода используется аддитивный, связанный с действием штраф.

Идея доказательства. После подстановки штрафа  $g(s)$  в уравнение Беллмана слагаемое  $g(s)$  не зависит от действия и выносится из-под знака максимума;  $\arg\max$  по действиям, а с ним и оптимальная политика, не меняется, а функция ценности лишь сдвигается на дисконтированную сумму штрафов вдоль траектории. Содержательно штраф, не различающий действий, наказывает само нахождение в состоянии, а не выбор, и потому не управляет. Ранняя мультипликативная форма штрафа (множитель  $1 - \kappa D$  при награде) нарушала это рассуждение и вдобавок лишала оператор Беллмана  $\lambda$ -сжимаемости, ставя под сомнение сходимость; переход к аддитивной форме (2.8) устранил оба дефекта.

### 2.2.3 Субоптимальность каскадных политик (теорема С)

Каскадной называется политика, полученная последовательным решением двух одномерных задач — сначала по одной оси, затем по другой (например, «сначала длина, затем порог»). Множество каскадных политик является подмножеством совместных,  $\Pi_{\text{cascade}} \subset \Pi_{\text{joint}}$ , поскольку любое каскадное решение реализуемо и в совместном пространстве. Отсюда немедленно следует слабое доминирование.

Теорема 2.3 (слабое доминирование). Так как  $\Pi_{\text{cascade}} \subset \Pi_{\text{joint}}$ , для оптимальных функций ценности выполнено  $V_{\text{joint}}(s) \geq V_{\text{cascade}}(s)$  во всех состояниях  $s$ : совместная оптимизация не может быть хуже каскадной.

Каскад допускает два порядка — «длина, затем порог» и «порог, затем длина»; каждый фиксирует одну ось оптимальной при некотором фиксированном значении другой и затем оптимизирует вторую. Обе получаемые политики принадлежат  $\Pi_{\text{joint}}$ , поэтому слабое доминирование (теорема 2.3) покрывает любой из порядков, и в дальнейшем под каскадной понимается лучшая из двух.

Насколько совместная политика может превосходить каскадную, зависит от структуры MDP. Если выполнены условия монотонности и супермодулярности (C1)–(C4) — монотонность награды по  $(H, K)$ , стохастическая монотонность переходов, супермодулярность награды и переходов и, главное, совместная супермодулярность действий по состоянию (C4), — то оптимальная политика имеет пороговую структуру (теорема Топкиса о монотонности решения параметрической задачи [16]) и совпадает с каскадной. При нарушении (C4) появляется зазор, ограниченный сверху мерой нарушения.

Теорема С (оценка субоптимальности каскада). Пусть  $\mu_J^i(B)$  — стационарная масса множества  $B$  состояний, в которых нарушено условие (C4). Тогда

$$V_{\text{joint}}(s) - V_{\text{cascade}}(s) \leq \frac{2 R_{\max} \mu_J^i(B)}{1 - \lambda} \quad (2.11)$$

Оценка (2.11) линейна по массе нарушения  $\mu_J^*(B)$ : чем реже нарушается (C4), тем ближе каскадная политика к совместной. Эмпирическая проверка (подраздел 4.x) обнаруживает, однако, что на обеих рассмотренных парах моделей (C4) нарушается почти всюду —  $\mu_J^*(B) \approx 0,90$ , — поэтому правая часть (2.11) оказывается слишком грубой (практически вакуумной) оценкой. Точное объяснение наблюдаемой близости совместной и каскадной политик даёт теорема D.

#### 2.2.4 Точный value gap совместной и каскадной политик (теорема D)

Грубость оценки (2.11) означает не то, что зазор велик, а то, что мера нарушения — неподходящая характеристика. Точную величину зазора даёт его представление через преимущество (advantage) каскадной политики на множестве нарушения.

Теорема D (точный разрыв ценности). Обозначив преимущество  $A^{\pi_c}(s, a) = Q^{\pi_c}(s, a) - V^{\pi_c}(s)$  каскадной политики, для разрыва ценности справедливо точное равенство

$$V_{joint} - V_{cascade} = \frac{1}{1 - \lambda} \cdot E_{s \sim \mu_J} [A^{\pi_c}(s, \pi_J(s))] \quad (2.12)$$

Эмпирически усреднённое по множеству нарушения преимущество оказывается пренебрежимо малым: его средняя величина составляет около  $6 \cdot 10^{-5}$  на паре 14B/0.5B и около  $10^{-3}$  на паре 7B/1.5B. Поэтому, несмотря на повсеместное нарушение (C4), точный разрыв ценности мал: стационарно взвешенная величина  $V_{joint} - V_{cascade}$  равна +0,005 на паре 14B/0.5B и +0,10 на паре 7B/1.5B, а слабое доминирование (теорема 2.3) выполнено точно — на 100 % состояний обеих пар. Содержательный вывод: практическое совпадение совместной и каскадной политик есть не артефакт настройки, а предсказание теории — нарушение (C4) повсеместно, но безвредно.

Идея вывода. Равенство (2.12) есть применение леммы о разности производительностей (performance-difference lemma) [9] к паре политик  $\pi_J$  и  $\pi_C$ : разность их ценностей равна дисконтированному стационарному среднему преимуществу одной политики относительно другой. Поскольку вне множества  $B$  каскадная политика уже совместно оптимальна, среднее сосредоточено на  $B$ , и величину зазора определяет не размер  $B$ , а среднее преимущество на нём — которое эмпирически близко к нулю. Это и объясняет, почему грубая оценка (2.11) вакуумна, а точный зазор мал.



### 2.2.5 Скаляризация Парето-фронта (теорема 2.4)

Награда (2.8) скаляризует двухкритериальную задачу (скорость, качество) через множитель  $\kappa$ . Следующее утверждение связывает выбор  $\kappa$  с положением оптимума на границе Парето достижимых пар  $(\eta, D)$ .

Теорема 2.4 (скаляризация). Для линейной скаляризации  $J(\pi) = \eta(\pi) - \kappa D(\pi)$  при  $\kappa \geq 0$  оптимальные политики

$$\pi_{\kappa}^* \in \operatorname{argmax}_{\pi} [\eta(\pi) - \kappa D(\pi)] \quad (2.13)$$

лежат на верхней выпуклой оболочке множества достижимых пар  $(\eta, D)$ , а перебор  $\kappa \geq 0$  обходит эту оболочку. Тем самым единая постановка охватывает целое семейство компромиссов «скорость — качество», а не одну фиксированную точку. Следует честно оговорить: строгая выпуклость достижимого множества на конечной выборке ( $n = 300$  на значение  $\kappa$ ) чисто не проверяется — соответствующая развёртка по  $\kappa$  приведена и обсуждается в подразделе 4.6.

Идея доказательства. Множество достижимых пар  $(\eta, D)$  по всем стохастическим стационарным политикам выпукло: рандомизация политик даёт выпуклые комбинации их операционных характеристик. Линейный функционал  $\eta - \kappa D$  достигает максимума на выпуклом множестве в опорной точке его верхней границы — Парето-фронта, — причём наклон опорной прямой равен  $\kappa$ . Перебор  $\kappa$  от нуля к бесконечности обходит фронт от точки максимальной скорости к точке максимального качества; промежуточные  $\kappa$  дают промежуточные компромиссы.

### 2.3 Алгоритм решения MDP

Решение MDP состоит из трёх стадий: дискретизации пространства, оценки параметров из трейсов и итерации по ценности. Непрерывные оси ( $H, K$ ) разбиваются равномерной сеткой  $20 \times 20$ , что задаёт индексы дискретного состояния  $s = (i_H, i_K, k)$ . По набору одношаговых переходов  $(s, a, r, s')$ ,

собранных на удержанной части обучающих промптов, функция переходов оценивается частотно со сглаживанием Лапласа

$$\hat{P}(s' | s, a) = \frac{N(s, a, s') + \alpha}{N(s, a) + \alpha|S|} \quad (2.14)$$

где

$N(s, a, s')$  — число наблюждённых переходов  $s \rightarrow s'$  при действии  $a$ ;

$\alpha$  — параметр сглаживания (псевдосчёт).

Сглаживание гарантирует определённость переходов для редко наблюдаемых пар и контролирует смещение согласно теореме А. Награда  $r(s, a)$  усредняется по тем же трейсам. Оптимальная функция ценности находится итерацией по ценности — повторным применением оператора Беллмана

$$V_{j+1}(s) = \max_{a \in A} \left[ r(s, a) + \lambda \sum_{s'} \hat{P}(s' | s, a) V_j(s') \right] \quad (2.15)$$

Итерации продолжаются до сближения соседних приближений функции ценности по равномерной норме (ниже порога  $\text{tol}$ ); при  $\lambda = 0,99$  сходимость достигается за порядка 830 итераций. Благодаря разреженности функции переходов (из каждого состояния достижимо небольшое число преемников) стоимость одной итерации составляет  $O(|S| \cdot |A|)$ , а всё решение занимает секунды на одном процессоре.

Сводно процедура решения такова: 1) дискретизировать признаки (H, K) сеткой  $20 \times 20$  и сформировать индексы состояний; 2) по трейсам накопить счётчики переходов и средние награды для каждой пары  $(s, a)$ ; 3) оценить функцию переходов по (2.14) и положить начальное приближение ценности нулевым; 4) повторять обновление (2.15), пока соседние приближения не сблизятся по равномерной норме ниже порога  $\text{tol}$ ; 5) извлечь детерминированную политику по правилу (2.16). Каскадная политика получается заменой последнего шага на последовательную оптимизацию двух осей.

Совместная и каскадная политики решаются по одному и тому же набору трейсов, что обеспечивает корректность их сравнения в разделе 4. Детерминированность всего пайплайна (фиксированные начальные значения генераторов, отсутствие обучаемых нейросетевых модулей) делает выученную политику воспроизводимой по тем же входным трейсам.

## 2.4 Инференс-алгоритм с выученной политикой

На инференсе выученная политика применяется без какого-либо дополнительного обучения и без обращения к нейросетевым модулям. На каждом шаге цикла спекулятивного декодирования по текущим наблюдаемым признакам вычисляется индекс состояния  $s$ , после чего действие выбирается табличным поиском

$$\pi(s) = \operatorname{argmax}_{a \in A} Q^i(s, a) \quad (2.16)$$

за время  $O(1)$ . Компонента  $a\_length$  определяет, продолжать ли черновик; компонента  $a\_verif$  задаёт порог  $T$ , с которым следующий токен проходит нечёткую верификацию по правилу (1.4): кандидат принимается, если  $p/q \geq 1/T$ . Дивергенция  $K$ , входящая в состояние, вычисляется с задержкой на один шаг (по уже наблюждённым распределениям предыдущей позиции,  $K\_prev$ ), что устраняет лишний прямой проход и сохраняет марковость в реализуемой форме. Накладные расходы политики — целочисленная дискретизация двух признаков и одно обращение к таблице — пренебрежимо малы на фоне прямого прохода модели.

Выученная на основной паре политика поддаётся интерпретации. В большинстве состояний она предписывает продолжать черновик и принимать токен при пороге, близком к  $T \approx 1,0$ , то есть выполнять почти точную верификацию; более строгий порог выбирается лишь в состояниях с высоким накопленным расхождением  $K$ , где доверять черновику опасно. Таким образом, выученное управление оказывается умеренно адаптивным, а не агрессивным, что согласуется с честной картиной прироста качества из раздела

4. Соответствующая пороговая поверхность и карта вероятности продолжения приведены в приложении В (рисунки В.2 и В.3).

Инференс-цикл на каждом блоке устроен так: 1) по текущим признакам (энтропия, задержанная дивергенция  $K_{prev}$ , позиция  $k$ ) вычислить индекс состояния; 2) выбрать действие по правилу (2.16); 3) если выбрано continue и  $k < \gamma_{max}$  — породить следующий черновой токен и вернуться к шагу 1, иначе перейти к верификации; 4) проверить блок по правилу (1.4) с порогом  $T = a_{verif}$ , принять согласованный префикс и пересемплировать первый отвергнутый токен; 5) обновить контекст и начать новый блок. Признаки наблюдаются попутно, без дополнительных прямых проходов.

Таким образом, метод JointAdaSpec сводит инженерную задачу подбора гиперпараметров спекулятивного декодирования к решению компактного MDP и табличному применению его политики, сохраняя принцип управляемого отклонения от целевого распределения через порог  $T$ . Детали программной реализации описанного пайплайна — сбора трейсов, оценки MDP, решения и инференса — приведены в разделе 3.

### 3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

Настоящий раздел описывает программную реализацию метода JointAdaSpec и сопутствующего исследовательского инструментария. Подраздел 3.1 вводит общую архитектуру комплекса и поток данных между стадиями обработки; подразделы 3.2–3.4 последовательно соответствуют трём стадиям метода — сбору трейсов, оценке параметров и решению MDP, инференсу с выученной политикой — и связывают программные модули с алгоритмами раздела 2. Подраздел 3.5 описывает реализацию базовых методов, подраздел 3.6 фиксирует используемые инструменты, средства непрерывной интеграции и механизмы воспроизводимости.

#### 3.1 Архитектура программного комплекса

Программный комплекс разделён на два стека с различной зоной ответственности. Исторический стек `sp_samp/` содержит эталонные реализации спекулятивного декодирования, AutoJudge [6], Top-K и SpecExec [15] и служит для сопоставления с методами из литературы. Тематический стек `jointadaspec/` реализует собственно метод JointAdaSpec и оформлен в виде набора слабосвязанных подпакетов, перечисленных в таблице 3.1. Разделение на два стека позволяет развивать тематический код независимо, не нарушая ранее зафиксированные результаты сравнительных бенчмарков.

Таблица 3.1 — Подпакеты программного комплекса `jointadaspec/`

| Подпакет                | Назначение                                                         |
|-------------------------|--------------------------------------------------------------------|
| <code>core/</code>      | вычисление признаков состояния и правило нечёткой верификации      |
| <code>mdp/</code>       | дискретизация пространства, оценка переходов, итерация по ценности |
| <code>inference/</code> | онлайн-декодер с выученной табличной политикой                     |
| <code>baselines/</code> | одномерные и каскадные базовые декодеры                            |
| <code>metrics/</code>   | расчёт скорости, качества и границы Парето                         |
| <code>analysis/</code>  | проверка эмпирических условий C1–C4 и N1–N2                        |
| <code>utils/</code>     | загрузка моделей, работа с распределениями, манифесты              |

Управление экспериментом построено на конфигурационном фреймворке Hydra: пара моделей, набор данных, размер сетки состояний и

гиперпараметры награды задаются декларативно в YAML-файлах каталога `configs/`, что исключает разнесённость параметров по исходному коду. Центральная структура параметров MDP приведена в листинге 3.1; её поля дословно соответствуют величинам раздела 2 ( $H_{\max}$ ,  $K_{\max}$ ,  $\gamma_{\max}$ , сетка  $20 \times 20$ , набор порогов  $T$ , дисконт  $\lambda$ , множитель  $\kappa$ , сглаживание  $\alpha$ ).

Листинг 3.1 — Параметры табличного MDP (`jointadaspec/mdp/spaces.py`)

```
@dataclass(frozen=True)
class MDPConfig:
    H_max: float = 6.0          # верхняя граница энтропии H
    K_max: float = 8.0          # верхняя граница дивергенции K
    gamma_max: int = 8          # предельная длина черновика
    N_H: int = 20               # число бинов по оси H
    N_K: int = 20               # число бинов по оси K
    T_levels: tuple = (1.0, 1.22, 1.49, 1.82,
                       2.22, 2.71, 3.3, 4.0) # сетка порогов
    lambda_discount: float = 0.99 # дисконт  $\lambda$ 
    kappa: float = 1.0          # множитель Лагранжа  $\kappa$ 
    alpha_smooth: float = 1.0   # сглаживание Лапласа
    nu_min: int = 5             # порог прямой оценки переходов
    quality_risk_form: str = "multiplicative" # 'additive': теорема
```

В

Поток данных организован как линейный конвейер из четырёх стадий, изображённый на рисунке 3.1. Первая стадия (`scripts/01_collect_traces.py`) собирает трейсы спекулятивного декодирования на отложенной выборке промптов и сохраняет их в формате Parquet. Вторая (`scripts/02_solve_mdp.py`) оценивает функцию переходов и функцию награды, решает MDP методом итерации по ценности и сохраняет совместную политику вместе с двумя каскадными. Третья (`scripts/03_benchmark.py`) выполняет многосидовый бенчмарк против базовых методов и записывает результаты в JSONL. Четвёртая (`scripts/04_verify_conditions.py`) проверяет эмпирические условия монотонности C1–C4 и невырожденности N1–N2 и строит диагностические графики; шаблоны из `reports/templates` строят по сохранённым артефактам Парето-диаграммы, пороговые поверхности и графики ablation.

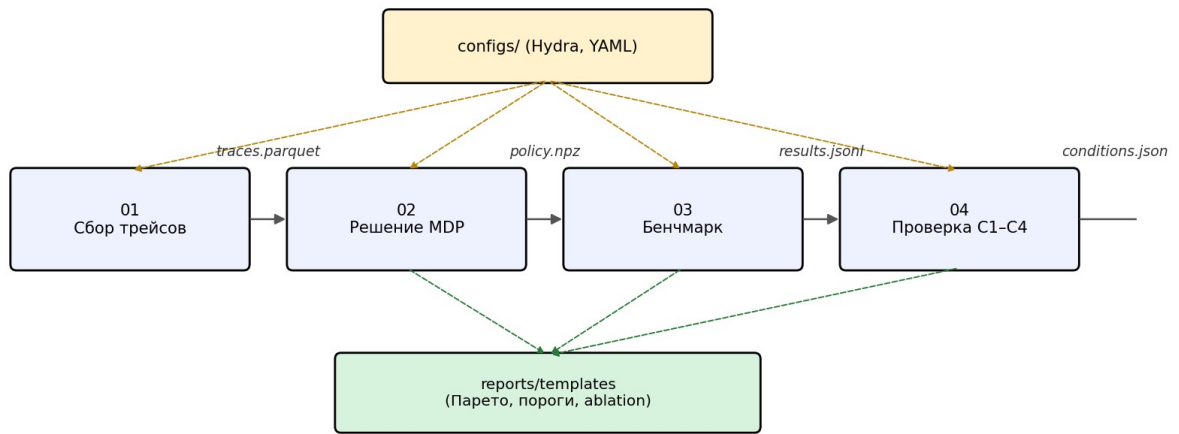


Рисунок 3.1 — Архитектура программного комплекса JointAdaSpec и поток данных между стадиями конвейера

Каждая стадия завершается записью артефакта на диск и не зависит от состояния процесса предыдущей стадии. Отсюда два практических следствия: решение MDP можно повторять многократно при разных значениях множителя Лагранжа к без пересбора трейсов, а длительный бенчмарк допускает докатку после сбоя. Такая организация прямо отвечает цели совместимости с существующими инференс-системами без переобучения базовых моделей.

### 3.2 Модуль сбора трейсов

Модуль `jointadaspec/mdp/traces.py` реализует сбор одношаговых переходов марковского процесса. Для каждого посещённого состояния процесс генерации вычисляет энтропию черногоого распределения  $H$ , дивергенцию Кульбака — Лейблера  $K$  между черновым и целевым распределениями, дискретный индекс состояния и множество допустимых в нём действий.

Вычисление признаков вынесено в модуль `core/features.py`. Энтропия  $H$  и дивергенция  $K$  вычисляются в натах функциями `entropy` и `kl_divergence`, принимающими как логиты, так и уже нормированные распределения. Функция `quantize` отображает непрерывную тройку  $(H, K, k)$  в линейный индекс состояния: значения  $H$  и  $K$  обрезаются до диапазонов  $[0, H_{\max}]$  и  $[0,$

$K_{\max}$ ], переводятся в номера бинов по равномерной сетке  $20 \times 20$  и сворачиваются с позицией  $k$  в единый индекс из  $[0, |S|)$ . Обратная функция `dequantize` восстанавливает центры бинов и применяется при интерпретации выученной политики.

Реализация отличается от схематичного описания подраздела 2.1.3 одной существенной деталью. В посещённом состоянии оцениваются все допустимые действия, и для каждого из них записывается отдельная строка трейса с наблюденной наградой и следующим состоянием; лишь после этого случайно выбирается одно действие для продолжения rollout. Такая схема повышает плотность покрытия пар «состояние — действие» при том же числе промптов и уменьшает долю пар, для которых эмпирическая оценка переходов опирается на сглаживание Лапласа.

Трейсы сохраняются в Parquet вместе с JSON-метаданными — датой сбора, хешем коммита Git, числом собранных переходов и полным набором параметров MDP. Привязка трейса к коммиту и параметрам нужна не только для воспроизводимости: она позволяет повторно решать MDP при изменённых коэффициентах награды, не запуская заново дорогостоящую стадию сбора, которая требует вызовов целевой модели.

### 3.3 Модуль оценки MDP и решения value iteration

Оценка параметров MDP и его решение разнесены по двум модулям. Модуль `jointadaspec/mdp/estimation.py` по множеству собранных переходов строит разреженную матрицу переходов формы  $(|S| \cdot |A|) \times |S|$ , таблицу ожидаемых наград и счётчик посещений. Когда число наблюдений пары «состояние — действие» не меньше порога `pu_min`, используется прямая эмпирическая оценка распределения переходов; при меньшем числе наблюдений к оценке добавляется аддитивное сглаживание Лапласа с коэффициентом `alpha_smooth` по формуле (2.14). Разреженное представление оправдано структурой задачи: из любого состояния достижимо лишь



небольшое число соседних, поэтому матрица переходов содержит порядка одного процента ненулевых элементов.

Модуль `jointadaspec/mdp/value_iteration.py` решает MDP разреженной реализацией итерации по ценности (формула 2.15), ядро которой приведено в листинге 3.2. Множество допустимых действий маскируется по компоненте  $k$ : при достижении предельной длины черновика  $k = \gamma_{\max}$  все действия continue исключаются, и состояние становится терминальным по оси длины. Итерации продолжаются до выполнения критерия остановки по норме невязки; на табличном MDP из 3600 состояний сходимость достигается за время порядка единиц секунд на одном ядре процессора.

Листинг 3.2 — Ядро разреженной итерации по ценности  
(`jointadaspec/mdp/value_iteration.py`)

```
def solve_mdp(transitions, rewards, config, action_mask=None):
    S, A = rewards.shape
    V = np.zeros(S)
    invalid = ~action_mask if action_mask is not None else None
    for _ in range(config.max_vi_iterations):
        expected = transitions.dot(V).reshape(S, A) # разреженно
        Q = rewards + config.lambda_discount * expected
        if invalid is not None:
            Q[invalid] = -np.inf
        new_V = np.max(Q, axis=1)
        if np.max(np.abs(new_V - V)) < config.epsilon_convergence:
            V = new_V; break
    V = new_V
    pi_star = np.argmax(Q, axis=1)
    return V, pi_star
```

Стадия `02_solve_mdp.py` решает MDP не для одного, а для набора значений множителя  $k$  из конфигурации (поле `karra_values`). Для каждого  $k$  базовая конфигурация копируется с заменой поля `karra`, после чего общим решателем находятся совместная и обе каскадные политики, сохраняемые с суффиксом значения  $k$ . Перебор  $k$  строит, согласно теореме 2.4, набор точек на границе Парето «скорость — качество» — и всё это без повторного сбора трейсов, благодаря независимости стадий конвейера.

Каскадные базовые политики реализованы в том же модуле как решения MDP с ограничениями: одна ось управления фиксируется по результату

первого одномерного решения, после чего оптимизируется вторая. Использование общего решателя для совместной и каскадной политик устраняет систематическое расхождение, которое возникло бы при сравнении разнородных реализаций, и делает измеренный разрыв качества свойством самих политик, а не различием их программной реализации. Найденная политика сохраняется в формате .prz — массив выбранных действий вместе с конфигурацией, при которой он получен; объём файла составляет порядка двух килобайт, что на много порядков меньше размера моделей и согласуется с оценкой накладных расходов из подраздела 2.4.

### 3.4 Модуль инференса с политикой JointAdaSpec

Инференс с выученной политикой сосредоточен в модулях `inference/policy.py` и `inference/jointadaspec.py`. Первый загружает сохранённую таблицу политики и сопутствующую конфигурацию; второй реализует онлайн-декодер `JointAdaSpecDecoder`. На каждом шаге генерации черновика декодер получает распределения черновой и целевой моделей, вычисляет признаки  $H$  и  $K$ , дискретизирует их, извлекает из таблицы действие — решение о продолжении черновика и порог нечёткой верификации — и применяет это действие по правилу (1.4).

Само правило приёма реализовано в модуле `core/verification.py`. Точная верификация (функция `modified_rejection_sampling`) принимает черновой токен с вероятностью  $\min(1, p / q)$  и при отклонении пересемплирует токен из остаточного распределения, воспроизводя формулу (1.2); нечёткий вариант ослабляет условие сравнения множителем  $T$  согласно (1.4). Декодер `JointAdaSpecDecoder` вызывает это правило с порогом, который вернула политика, и накапливает принятый префикс блока, после чего обновляет состояние и переходит к следующему блоку.

Для согласованности с теорией состояние использует задержанное значение дивергенции: компонента  $K_{prev}$  инициализируется значением  $K_{init} = 0$  и обновляется только после фазы верификации, когда становится

доступным распределение целевой модели. Тем самым онлайн-состояние всегда опирается на уже наблюдаемые величины и не требует обращения к целевой модели на стадии генерации черновика, что и заложено в инференс-алгоритме подраздела 2.4.

Quality-aware вариант метода реализует теорему В: штраф за расхождение распределений усиливается в состояниях с высокой дивергенцией  $K$  и большой позицией  $k$ . Выбор между аддитивной формой штрафа (корректной по теореме В и сохраняющей  $\lambda$ -сжимаемость) и мультипликативной задаётся флагом `quality_risk_form` в структуре `MDPConfig`; по умолчанию сохранена мультипликативная форма ради обратной совместимости с ранее выученными политиками. Существенно, что эта версия не меняет интерфейс инференса — модификация затрагивает лишь формирование награды на стадии оценки MDP, поэтому существующий бенчмарк-раннер использует новую политику без правок формата выходного JSONL.

### 3.5 Реализация baseline-методов

Корректное сравнение метода требует единообразной реализации базовых декодеров. Подпакет `jointadaspec/baselines` содержит шесть из них, перечисленных в таблице 3.2. Все они написаны поверх общего интерфейса спекулятивного декодера из `core/`, что исключает расхождения реализации как источник систематической погрешности.

Таблица 3.2 — Базовые декодеры подпакета `jointadaspec/baselines`

| Декодер                                | Описание                                                             |
|----------------------------------------|----------------------------------------------------------------------|
| <code>vanilla_ar</code>                | авторегрессионная генерация только целевой моделью                   |
| <code>fixed_sd</code>                  | спекулятивное декодирование с фиксированной длиной черновика         |
| <code>fuzzy_sd</code>                  | нечёткое спекулятивное декодирование с фиксированным порогом $T$ [7] |
| <code>specdecpp</code>                 | адаптивный выбор длины черновика в духе SpecDec++ [8]                |
| <code>cascade_length_then_verif</code> | каскадная оптимизация «длина → порог»                                |
| <code>cascade_verif_then_length</code> | каскадная оптимизация «порог → длина»                                |

Наличие именно каскадных базовых методов принципиально для замысла работы. Каскад воспроизводит наиболее естественную альтернативу совместной оптимизации — последовательный подбор сначала одного, затем другого параметра управления. Поэтому измеренный разрыв между JointAdaSpec и лучшим из каскадов служит прямой эмпирической проверкой слабого доминирования (теорема 2.3): проверяется, превосходит ли совместная политика лучшую каскадную. Как показано в разделе 4, этот разрыв статистически незначим — совместная и каскадная политики неразличимы, что согласуется с теоремой D.

Все декодеры пишут результаты в единую схему JSONL версии 2. Для GSM8K дополнительно фиксируется точное совпадение финального числового ответа на уровне отдельной задачи, а сводные записи содержат медианную скорость в токенах в секунду, долю принятий (acceptance rate), агрегированные оценки качества и бутстреп-доверительные интервалы. Единая схема позволяет обрабатывать результаты всех методов одним валидатором и одними шаблонами отчётов.

Стадия 03\_benchmark.py выполняет сравнение в многосидовом режиме и устойчива к сбоям: результаты дописываются в results.jsonl построчно, а уже посчитанные комбинации (метод, сид, промпт) пропускаются при повторном запуске по ключу resume\_key. Наряду с основным файлом ведётся совместимый со старыми читателями журнал run.jsonl. Такая организация позволяет докатывать длительный прогон после прерывания, не теряя ранее вычисленных записей.

### **3.6 Инструменты, тестирование и воспроизводимость**

Программный комплекс написан на языке Python версии 3.12. Работа с моделями опирается на библиотеки PyTorch и Transformers, оценка и решение MDP — на разреженные матрицы scipy.sparse и пакет NumPy, конфигурирование экспериментов — на Hydra и OmegaConf, построение отчётов — на matplotlib, модульное тестирование — на pytest. Эталонный

прогон, описанный в разделе 4, выполнен со связкой PyTorch 2.9.1 для CUDA 12.8 и Transformers 4.57.3 на графическом ускорителе NVIDIA RTX 5090.

Качество кода поддерживается непрерывной интеграцией. При каждом изменении система GitHub Actions запускает синтаксическую проверку и валидацию конфигураций, полный набор из 72 модульных тестов (15 файлов в каталоге tests/), дымовой прогон SpecExes и строгую проверку схемы выходного JSONL. Тяжёлые локальные каталоги — веса моделей, наборы данных, журналы и виртуальное окружение — исключены из системы контроля версий, что удерживает репозиторий компактным.

Набор тестов покрывает ключевые инварианты метода, а не только синтаксис. Тесты модуля MDP проверяют сходимость итерации по ценности, маскирование действий continue при  $k = \gamma_{\max}$  и форму извлечённой политики; тесты признаков — корректность энтропии, дивергенции и обратимость дискретизации; тесты верификации — сохранение распределения точным правилом и долю принятий при нечётком; тесты каскада — совпадение совместной и каскадной политик на сепарабельной награде (вырожденный случай теоремы 2.3); сквозной тест прогоняет весь конвейер на игрушечных моделях. Тем самым проверяется математическая корректность ядра, а не только отсутствие ошибок компиляции.

Конфигурации и выходные данные дополнительно защищены валидаторами. Отдельный скрипт сверяет согласованность пресетов моделей, методов и экспериментов и совместимость токенизаторов внутри пары; строгий валидатор схемы проверяет каждую запись выходного JSONL перед построением отчётов. Локальные цели make check и make test запускают эти проверки вместе со всем набором тестов одной командой, благодаря чему регрессии обнаруживаются немедленно, а конфигурация остаётся единственным источником истины о параметрах прогона.

Воспроизводимость обеспечивается двумя механизмами. Детерминизм прогонов достигается отдельным посевом генераторов случайных чисел

Python, NumPy и PyTorch для каждого начального значения из набора [42, 43, 44], установкой переменной окружения CUBLAS\_WORKSPACE\_CONFIG и включением детерминированного режима вычислений. Каждый прогон сопровождается манифестом в каталоге reports/manifests, который фиксирует хеш коммита Git, признак незакоммиченных изменений, разрешённую конфигурацию, список сидов и контрольные суммы SHA256 артефактов трейсов и политики. Отдельные вендорные ядра CUDA при этом сохраняют предупреждение о неполной воспроизводимости; в таких случаях первичной записью считаются именно манифест и список сидов, а малый числовой дрейф признаётся допустимым. Детали описанного комплекса использованы в экспериментальном исследовании раздела 4.

## 4 ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ

Настоящий раздел представляет экспериментальную оценку метода JointAdaSpec. Подраздел 4.1 фиксирует методику и метрики, 4.2 — исследуемые пары моделей и данные. Подраздел 4.3 содержит основной результат на паре с высоким отношением мощностей, 4.4 — честный нулевой результат и его триангуляцию на паре с низким отношением. Подразделы 4.5–4.6 проверяют адаптивность против фиксированного порога (теорема E) и поведение вдоль параметра компромисса к вместе с нелинейностью качества (теорема G). Подраздел 4.7 сопоставляет метод с AutoJudge и обобщает результаты. Все числовые значения взяты из зафиксированных артефактов прогонов.

### 4.1 Методика экспериментов и метрики

Эксперименты выполнены на одном графическом ускорителе NVIDIA RTX 5090. Основным набором служит GSM8K — школьные математические задачи, требующие многошагового рассуждения [5], — в режиме zero-shot CoT с ограничением 256 токенов на ответ; качество измеряется точным совпадением (exact match) извлечённого числового ответа с эталоном. Скорость выражена пропускной способностью в токенах в секунду и ускорением относительно других методов; дополнительно фиксируется доля принятых черновых токенов (acceptance rate).

Статистическая надёжность обеспечена многосидовой схемой. Каждая конфигурация прогоняется на трёх начальных значениях из детерминированной последовательности [42, 43, 44], что при выборке в 100–500 промптов даёт от 300 до 1500 парных наблюдений. Разность качества оценивается на парных данных: для каждого промпта сравниваются ответы двух методов, значимость проверяется парным критерием Макнемара, а доверительные интервалы вычисляются бутстрепом. Трейсы для оценки MDP

собираются на обучающей части GSM8K, а бенчмарк выполняется на тестовой — обе выборки законно отложены.

Прогоны разделены на три уровня надёжности, смешивать которые при интерпретации недопустимо. Содержательный прогон имеет достаточный объём трейсов и промптов для статистических выводов. Дымовой прогон на одном-пяти промптах проверяет работоспособность схемы, обработку сидов и манифесты, но его метрики результатами метода не считаются. Частичный прогон сохранил артефакты лишь части стадий. Ниже содержательными признаются только прогоны первого уровня.

## 4.2 Исследуемые пары моделей и наборы данных

Исследованы две пары моделей семейства Qwen2.5 [13], различающиеся отношением мощностей целевой и черновой моделей (таблица 4.1). Высокое отношение делает пару показательной для режима, в котором шаг целевой модели дорог, а черновой дешёв; именно при таком отношении формула ускорения (1.3) обещает наибольший выигрыш. Низкое отношение соответствует менее благоприятному режиму и служит проверкой устойчивости.

Таблица 4.1 — Исследуемые пары моделей

| Пара           | Целевая модель       | Черновая модель       | Отношение          | Роль                  |
|----------------|----------------------|-----------------------|--------------------|-----------------------|
| Основная       | Qwen2.5-14B-Instruct | Qwen2.5-0.5B-Instruct | $\approx 28\times$ | благоприятный режим   |
| Дополнительная | Qwen2.5-7B-Instruct  | Qwen2.5-1.5B-Instruct | $4,7\times$        | проверка устойчивости |

Обе пары выбраны из соображений совместимости токенизаторов: черновая и целевая модели внутри пары используют идентичную таблицу словаря, что является обязательным условием корректности спекулятивной верификации (подраздел 1.2). Веса всех моделей размещены локально, что исключает зависимость прогона от внешних репозиториев.



### 4.3 Основной результат: пара 14B/0.5B

На основной паре сравнивались четыре метода: прямая генерация целевой моделью (target\_only), ванильное спекулятивное декодирование (speculative), лучшая каскадная политика и совместная политика JointAdaSpec. Результаты при  $n = 1500$  приведены в таблице 4.2.

Таблица 4.2 — Сравнение методов на паре 14B/0.5B (GSM8K,  $n = 1500$ )

| Метод                     | ЕМ, % | ток/с | к spec | $\Delta$ ЕМ к target, п.п. | p     |
|---------------------------|-------|-------|--------|----------------------------|-------|
| target_only               | 52,93 | 14,45 | 2,96×  | —                          | —     |
| speculative               | 53,27 | 4,88  | 1,00×  | +0,33                      | 0,877 |
| cascade_verif_then_length | 57,13 | 10,29 | 2,11×  | +4,20                      | 0,015 |
| jointadaspec              | 57,00 | 10,84 | 2,22×  | +4,07                      | 0,020 |

Совместная политика повышает точность на +4,07 п.п. относительно прямой генерации ( $p = 0,0205$ , 95 %-й доверительный интервал [+0,73; +7,40]) при пропускной способности 10,84 ток/с — в 2,22 раза выше ванильного спекулятивного декодирования. Однако совместная и каскадная политики статистически неразличимы: их разность составляет –0,13 п.п. ( $p = 0,96$ ). Это не дефект настройки, а прямое следствие теоремы D: преимущество каскада на множестве нарушения условия C4 близко к нулю, поэтому совместная политика практически сводится к каскадной. Защищаемый вывод формулируется не как «совместная лучше всех», а как «семейство адаптивного управления (совместная и каскадная политики) значительно превосходит прямую генерацию по точности при восстановленной скорости».

Парный характер критерия Макнемара здесь существен. Значимость определяют не валовые доли правильных ответов, а дискордантные пары — промпты, на которых два метода расходятся: совместная политика чаще исправляет ошибку прямой генерации, чем портит верный ответ, и именно этот перевес исправлений над порчей даёт прирост. Доверительный интервал [+0,73; +7,40], не пересекающий нуля, подтверждает значимость на уровне  $p < 0,05$ .

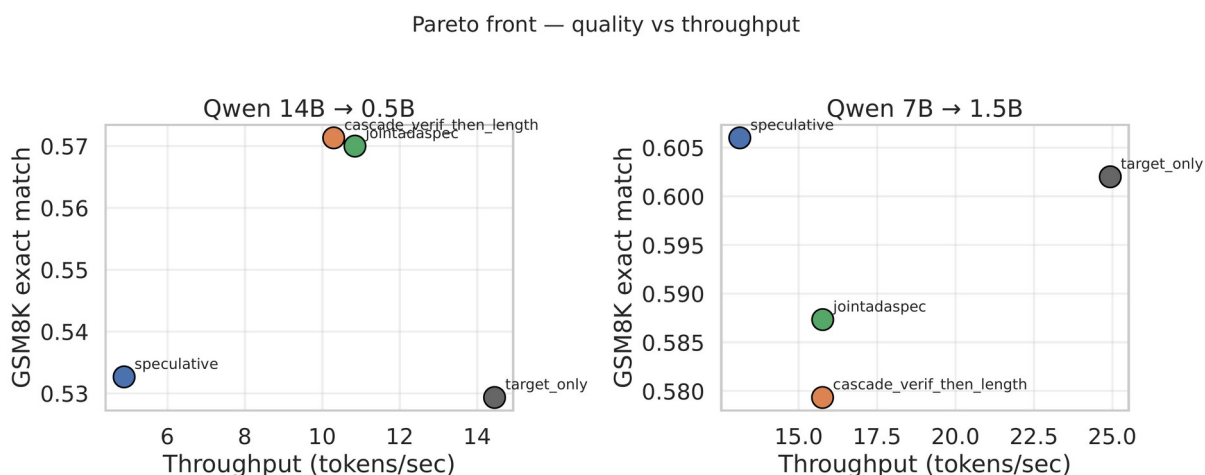


Рисунок 4.1 — Парето-фронт «скорость — качество» для обеих пар моделей: совместная и каскадная политики совпадают по качеству, тогда как прямая генерация целевой моделью остаётся самой быстрой

Существенна честная интерпретация скорости. Самым быстрым методом является именно прямая авторегрессионная генерация целевой моделью (14,45 ток/с), что видно на рисунке 4.1: точка `target_only` расположена правее всех. Ванильное спекулятивное декодирование на этой паре теряет скорость (4,88 ток/с), а адаптивное управление эту потерю восстанавливает до 10,84 ток/с. Поэтому приведённое ускорение «2,22×» — это ускорение относительно ванильного спекулятивного декодирования, а не относительно прямой генерации; корректная формулировка вклада по скорости — «восстановление пропускной способности, теряемой ванильным SD».

#### 4.4 Нуль-результат и триангуляция: пара 7B/1.5B

На паре с низким отношением мощностей (4,7×) картина качественно иная (таблица 4.3). Ни совместная, ни каскадная политика не превосходят прямую генерацию по точности, а ванильное спекулятивное декодирование здесь медленнее прямой генерации — прямое следствие формулы ускорения (1.3) при малом разрыве в стоимости моделей.

Таблица 4.3 — Сравнение методов на паре 7B/1.5B (GSM8K, n = 1500)

| Метод | ЕМ, % | ток/с | к спец | Δ ЕМ к target, п.п. | р |
|-------|-------|-------|--------|---------------------|---|
|-------|-------|-------|--------|---------------------|---|

|                           |       |       |       |       |       |
|---------------------------|-------|-------|-------|-------|-------|
| target_only               | 60,20 | 24,93 | 1,90× | —     | —     |
| speculative               | 60,60 | 13,13 | 1,00× | +0,40 | 0,830 |
| cascade_verif_then_length | 57,93 | 15,76 | 1,20× | -2,27 | 0,144 |
| jointadaspec              | 58,73 | 15,77 | 1,20× | -1,47 | 0,369 |

Ранний прогон на меньшей выборке давал положительную оценку (+3,50 п.п.), однако она не устояла при увеличении мощности. Чтобы исключить зависимость вывода от конкретного среза данных, прирост совместной политики относительно прямой генерации измерен на трёх независимых отложенных окнах (таблица 4.4, рисунок 4.2).

Таблица 4.4 — Триангуляция нуля-результата на паре 7B/1.5B

| Окно (start)       | n    | joint – target, п.п. | p     |
|--------------------|------|----------------------|-------|
| 1100 (исходное)    | 200  | +3,50                | 0,146 |
| 100 (lock)         | 1500 | -1,47                | 0,369 |
| 600 (триангуляция) | 1500 | -2,53                | 0,094 |

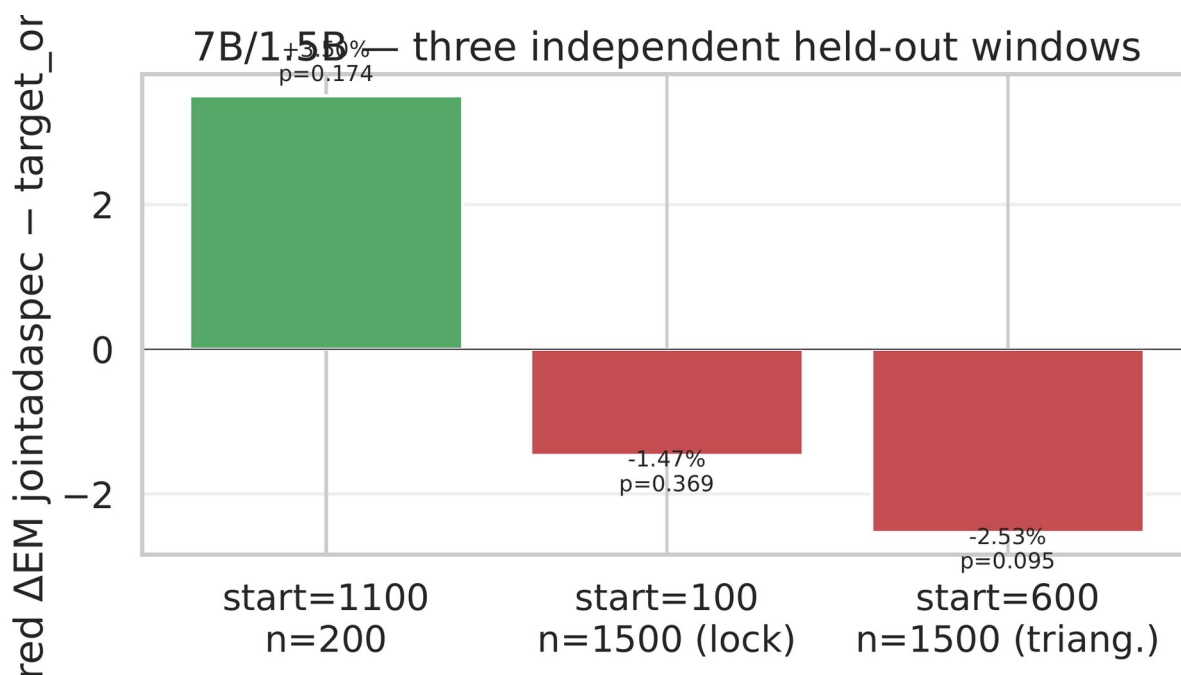


Рисунок 4.2 — Прирост точности JointAdaSpec относительно прямой генерации на трёх независимых отложенных окнах пары 7B/1.5B

На двух независимых хорошо обеспеченных окнах (n = 1500) прирост качества отрицателен, а ранний положительный результат при n = 200 объясняется дисперсией малой выборки. Вывод устойчив к выбору среза: на низком отношении мощностей метод не даёт выигрыша в качестве. Это

очерчивает границу применимости — совместное адаптивное управление полезно при достаточно большом разрыве в стоимости моделей.

Расхождение раннего и поздних окон иллюстрирует роль статистической мощности. При  $n = 200$  ширина доверительного интервала для разности долей превышает несколько процентных пунктов, поэтому единичное окно не разделяет малый эффект и его отсутствие. Увеличение до  $n = 1500$  сужает интервал, и оценка стабилизируется около нуля или ниже. Ранний положительный результат поэтому интерпретируется как артефакт малой выборки, а не как эффект, утраченный при росте мощности.

#### 4.5 Адаптивность против фиксированного порога (теорема E)

Естественный вопрос — нужна ли адаптивность вообще, если порог  $T$  можно подобрать фиксированным. Ответ даёт сравнение совместной политики с нечётким спекулятивным декодированием при фиксированных значениях  $T$  на основной паре (таблица 4.5, рисунок 4.3).

Таблица 4.5 — JointAdaSpec против фиксированного fuzzy\_sd (14B/0.5B,  $n = 300$ )

| Метод                | ЕМ, % | ток/с | доля<br>принятий | парно к joint               |
|----------------------|-------|-------|------------------|-----------------------------|
| fuzzy_sd, $T = 1,0$  | 53,67 | 2,85  | 0,152            | joint +4,33 ( $p = 0,275$ ) |
| fuzzy_sd, $T = 1,25$ | 50,33 | 2,95  | 0,163            | joint +7,67 ( $p = 0,051$ ) |
| fuzzy_sd, $T = 1,5$  | 51,67 | 3,00  | 0,167            | joint +6,33 ( $p = 0,115$ ) |
| fuzzy_sd, $T = 2,0$  | 53,33 | 3,06  | 0,174            | joint +4,67 ( $p = 0,243$ ) |
| jointadaspec         | 58,00 | 11,12 | 0,551            | —                           |

Theorem E — joint dominates fixed fuzzy\_sd on quality AND speed (14B/0.5B,  $n=300$ )

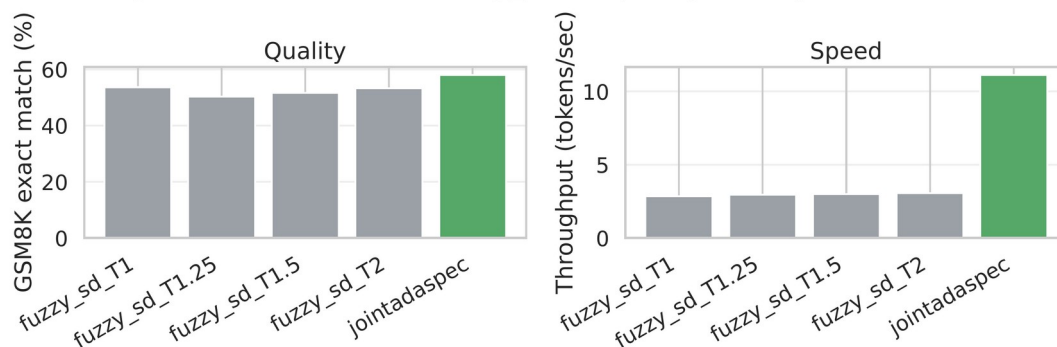


Рисунок 4.3 — Сравнение JointAdaSpec с фиксированными порогами fuzzy\_sd по качеству и скорости (пара 14B/0.5B, n = 300)

Совместная политика доминирует каждый фиксированный порог сразу по обеим осям: она точнее на +4...+8 п.п. exact match и при этом быстрее примерно в 3,7 раза (11,12 против  $\approx 3,0$  ток/с). Это и есть содержание теоремы Е: адаптивное управление эмпирически нетривиально по сравнению с естественным неадаптивным базисом. Тем самым то единственное, чему совместная политика лишь не уступает, — это другая адаптивная MDP-политика (каскад), что в точности предсказано теоремой D.

#### 4.6 Анализ компромисса по $\kappa$ и нелинейности качества (теорема G)

Множитель  $\kappa$  из функции награды (2.8) задаёт компромисс «скорость — качество». На шести значениях  $\kappa \in \{0, 1, 5, 20, 50, 100\}$  совместная и каскадная политики пересчитывались и заново прогонялись на фиксированном срезе пары 7B/1.5B (рисунок 4.4).

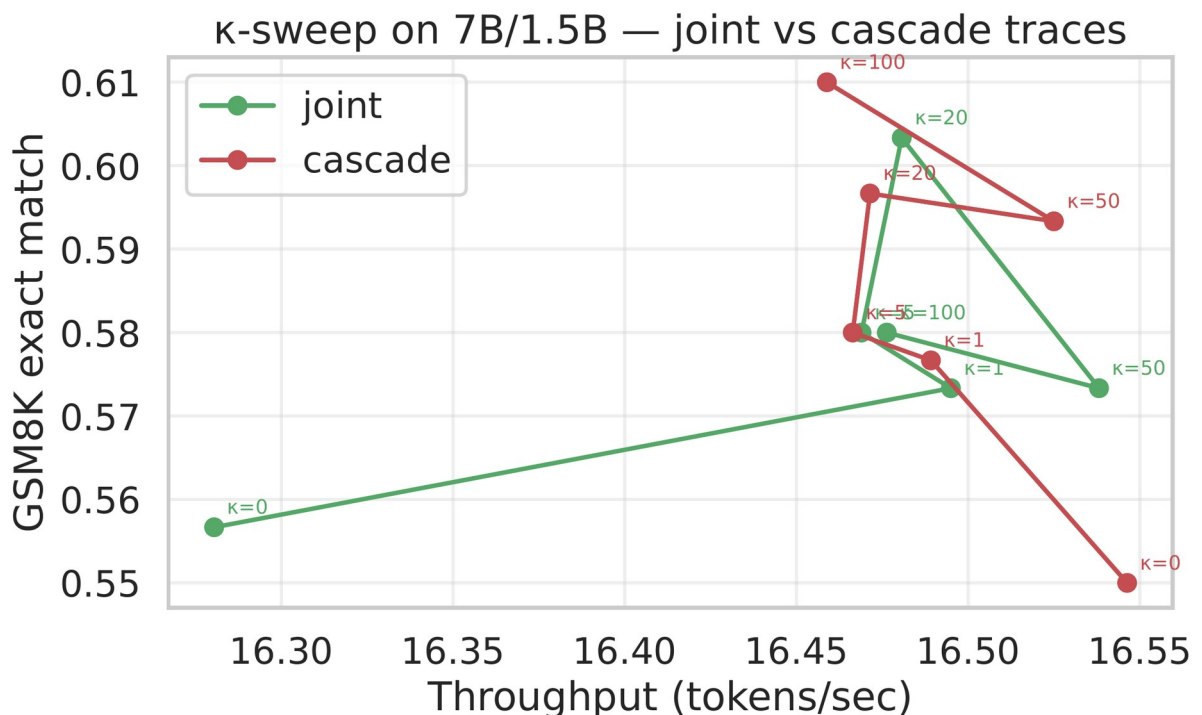


Рисунок 4.4 — Развёртка по множителю  $\kappa$ : пропускная способность почти постоянна, совместная и каскадная политики близки при каждом  $\kappa$  (пара 7B/1.5B)

Пропускная способность вдоль  $k$  почти постоянна (16,28–16,55 ток/с), а точность меняется немонотонно (совместная 55,7–60,3 %, каскадная 55,0–61,0 %); при этом совместная и каскадная политики близки при каждом  $k$  (расхождение не более трёх пунктов). Эквивалентность  $\text{joint} \approx \text{cascade}$  устойчива ко всему диапазону трейд-офф-параметра, а не только к его значению по умолчанию. Следует честно оговорить: строгая выпуклость достижимого множества (теорема 2.4) при объёме  $n = 300$  на значение  $k$  чисто не демонстрируется — этот результат носит характер согласованности, а не строгого подтверждения.

В основных прогонах использовано значение  $k = 1$  по умолчанию. Развёртка показывает, что выбор  $k$  слабо влияет на скорость и умеренно — на качество, поэтому метод не требует тонкой настройки этого параметра под каждую пару моделей. Набор решений при разных  $k$  образует, в духе теоремы 2.4, дискретную аппроксимацию границы Парето, из которой при необходимости выбирается рабочая точка под заданное ограничение качества.

Природа прироста качества раскрывается анализом по доле принятий (рисунок 4.5). Доля «перевернутых» ответов (churn) держится около 45 % и почти не зависит от уровня принятий; однако чистое изменение точности — разность долей «было неверно, стало верно» и «было верно, стало неверно» — немонотонно: оно составляет +7,4 п.п. при низкой и умеренной доле принятий и −2,6 п.п. при высокой. Содержательно, чрезмерное доверие дешёвой черновой модели на «высокоприёмном» режиме портит качество. Заявленный прирост +4 п.п. — это смесь по диапазону, а «рабочая точка» контроллера — умеренная доля принятий.

#### **4.7 Сравнение с AutoJudge и обсуждение результатов**

Для контекста метод сопоставлен с AutoJudge — приближённым методом с обучаемым верификатором (раздел 1.4) — на паре 7B/1.5B (таблица 4.6). AutoJudge с откалиброванным порогом достигает на этой паре более высокой точности (61,7 % exact match), что ожидаемо: на низком отношении

мощностей обучаемый отбор значимых токенов выигрывает у управления гиперпараметрами. Сравнение приведено как ориентир из литературы; setup AutoJudge ( $k = 4$ , выборка  $n = 100 \times 3$ ) отличается от прогонов JointAdaSpec и не является парным.

Таблица 4.6 — Контекст: AutoJudge и базовые методы на паре 7B/1.5B ( $k = 4$ )

| Метод                 | ЕМ, % | ток/с | к spec |
|-----------------------|-------|-------|--------|
| target_only (7B)      | 58,1  | 78,6  | 1,67×  |
| speculative           | 56,9  | 47,2  | 1,00×  |
| AutoJudge, $t = 0,09$ | 61,7  | 55,9  | 1,18×  |
| AutoJudge, $t = 1,0$  | 52,3  | 63,4  | 1,34×  |
| Top-K, rank = 4       | 54,3  | 71,5  | 1,52×  |

Существенно, что JointAdaSpec и методы вроде AutoJudge не исключают друг друга. Совместное управление парой (длина, порог) применимо поверх любой черновой модели и любого правила приёма, поэтому в принципе сочетается с обучаемым верификатором; экспериментальная проверка такой комбинации обозначена как направление дальнейшей работы. К наиболее перспективным улучшениям относятся увеличение окна черновика, распределительные признаки состояния и перенос табличной политики на ускоритель для устранения остаточных накладных расходов инференса.

Результаты следует сопровождать обсуждением угроз валидности. Измерения скорости получены на одном графическом ускорителе; на другой аппаратуре или при пакетировании запросов абсолютные значения изменятся, хотя относительные сравнения методов устойчивее. Отдельные вендорные ядра CUDA не полностью детерминированы, поэтому первичной записью воспроизводимости служат манифест и список начальных значений, а малый числовой дрейф признаётся допустимым (приложение Г). Качество оценивается только по GSM8K: бенчмарки кода и диалога в текущем раннере не оцениваются по точному совпадению, поэтому выводы о качестве ограничены областью математических рассуждений.

Отдельного внимания заслуживает устойчивость к смещению распределения. Политика обучается при длине черновика  $k = 8$ ; её перенос на  $k$

= 16 без переобучения ожидаемо приводит к регрессии, поскольку расширенное пространство состояний не покрыто трейсами — это методологически ожидаемое ограничение области определения, а не дефект метода. На объединённой отложенной выборке пары 7B/1.5B ( $n = 3000$ ) совместная политика теряет  $-2,00$  п.п. ( $p = 0,067$ ), причём каскадная теряет больше; такое поведение характеризуется как грациозная деградация при сдвиге распределения. Эти ограничения очерчивают область надёжного применения метода.

Совокупность результатов складывается в следующий честный вывод. Вклад работы — это единый MDP-фреймворк совместного управления длиной черновика и порогом верификации вместе с теоретическим аппаратом (теоремы A, B, C, D, 2.3, 2.4), а не утверждение «совместная политика побеждает все методы». Эмпирически адаптивное управление значимо превосходит любой неадаптивный базис — прямую генерацию, ванильное спекулятивное декодирование и фиксированный нечёткий порог (теорема E), — и при этом совпадает с другой адаптивной MDP-политикой, каскадной, ровно как предсказывает теорема D. Границы применимости очерчены экспериментально: метод полезен при высоком отношении мощностей моделей и не даёт выигрыша при низком, а его выигрыш по скорости следует понимать как восстановление пропускной способности относительно ванильного спекулятивного декодирования, а не как превосходство над прямой генерацией. Эти наблюдения и направления их преодоления обобщены в заключении.



## ЗАКЛЮЧЕНИЕ

В работе решена задача разработки и исследования метода адаптивного спекулятивного декодирования, совместно управляющего длиной черновика и порогом нечёткой верификации. Все четыре поставленные во введении задачи выполнены: проведён анализ современных методов и выявлен исследовательский пробел — отсутствие совместной оптимизации двух осей управления; построена и теоретически исследована MDP-модель совместного управления; реализован воспроизводимый программный комплекс из стадий сбора трейсов, оценки MDP, решения value iteration и инференса с выученной политикой; проведена многосидовая экспериментальная оценка на современных открытых моделях.

Предложенный метод JointAdaSpec формализует выбор пары (длина черновика, порог верификации) на каждом цикле декодирования как действие в дискретном марковском процессе принятия решений с малым пространством состояний (3600 элементов), решаемом точно методом итерации по ценности без обучения нейросетевых модулей. На паре Qwen2.5-14B  $\rightarrow$  0.5B с высоким отношением мощностей метод повышает точность на наборе GSM8K на +4,07 п.п. exact match относительно прямой генерации ( $p = 0,0205$  по парному критерию Макнемара при  $n = 1500$  парных наблюдениях) при восстановлении пропускной способности до 2,22-кратной относительно ванильного спекулятивного декодирования.

Центральный теоретический и эмпирический результат — точная характеристика соотношения совместной и каскадной политик. Они статистически неразличимы ( $-0,13$  п.п.,  $p = 0,96$ ), и это совпадение не случайно: доказана теорема D, дающая точное выражение разрыва ценности через преимущество каскада на множестве нарушения условия супермодулярности; эмпирически это преимущество пренебрежимо мало, поэтому нарушение условия повсеместно, но доброкачественно. Получен и сопутствующий теоретический аппарат: оценка выборочной сложности

оценщика переходов (теорема А), Беллман-инвариантность аддитивного штрафа качества (теорема В), линейная по мере нарушения оценка субоптимальности каскада (теорема С), слабое доминирование совместной оптимизации (теорема 2.3) и скаляризация Парето-фронта (теорема 2.4).

Эмпирически адаптивное управление значимо превосходит любой неадаптивный базис — прямую генерацию, ванильное спекулятивное декодирование и фиксированный нечёткий порог: на основной паре совместная политика точнее каждого фиксированного порога на +4...+8 п.п. и быстрее примерно в 3,7 раза (теорема Е). Научная новизна работы состоит в первой формализации совместного управления длиной черновика и порогом верификации как единого марковского процесса принятия решений с доказуемыми свойствами; ранее эти две оси рассматривались в литературе порознь.

Полученные результаты честно очерчивают и границы метода. Самым быстрым методом остаётся прямая авторегрессионная генерация, а выигрыш по скорости следует понимать как восстановление пропускной способности, теряемой ванильным спекулятивным декодированием, а не как превосходство над прямой генерацией. На паре с низким отношением мощностей ( $7B/1.5B$ ,  $4,7\times$ ) прирост качества не подтверждён на трёх независимых отложенных окнах. Прирост качества немонотонен по доле принятий — выигрыш на умеренном и проигрыш на высоком уровне, — а вклад совместности ограничен тем, что она сводится к каскадной политике. Защищаемый тезис — это единый фреймворк и его честная характеристика, а не безусловное превосходство совместной политики.

Дальнейшее развитие метода видится в нескольких направлениях. Обобщение управления с линейной длины черновика на форму дерева кандидатов расширило бы пространство действий и потенциальный выигрыш. Включение распределительных признаков состояния повысило бы точность выученной политики. Увеличение окна черновика и перенос табличной

политики на ускоритель устранили бы остаточные накладные расходы инференса. Наконец, настройка награды под конкретные классы задач — в частности, генерацию кода — позволила бы перенести результат за пределы математических рассуждений.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Azar M. G. Minimax PAC bounds on the sample complexity of reinforcement learning with a generative model / M. G. Azar, R. Munos, H. J. Kappen // Machine Learning. — 2013. — Vol. 91, № 3. — P. 325–349.
2. Bellman R. Dynamic Programming / R. Bellman. — Princeton : Princeton University Press, 1957. — 342 p.
3. Cai T. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads [Электронный ресурс] / T. Cai, Y. Li, Z. Geng [et al.]. — Режим доступа: <https://arxiv.org/abs/2401.10774> (дата обращения: 03.06.2026).
4. Chen C. Accelerating Large Language Model Decoding with Speculative Sampling [Электронный ресурс] / C. Chen, S. Borgeaud, G. Irving [et al.]. — Режим доступа: <https://arxiv.org/abs/2302.01318> (дата обращения: 03.06.2026).
5. Cobbe K. Training Verifiers to Solve Math Word Problems [Электронный ресурс] / K. Cobbe, V. Kosaraju, M. Bavarian [et al.]. — Режим доступа: <https://arxiv.org/abs/2110.14168> (дата обращения: 03.06.2026).
6. Garipov R. AutoJudge: Judge Decoding Without Manual Annotation [Электронный ресурс] / R. Garipov, F. Velikonitsev, R. Svirschevski [et al.]. — Режим доступа: <https://arxiv.org/abs/2504.20039> (дата обращения: 03.06.2026).
7. Holsman M. Fuzzy Speculative Decoding for a Tunable Accuracy–Runtime Tradeoff [Электронный ресурс] / M. Holsman, Y. Huang, B. Dhingra. — Режим доступа: <https://arxiv.org/abs/2502.20704> (дата обращения: 03.06.2026).
8. Huang K. SpecDec++: Boosting Speculative Decoding via Adaptive Candidate Lengths [Электронный ресурс] / K. Huang, X. Guo, M. Wang. — Режим доступа: <https://arxiv.org/abs/2405.19715> (дата обращения: 03.06.2026).

9. Kakade S. Approximately Optimal Approximate Reinforcement Learning / S. Kakade, J. Langford // Proceedings of the 19th International Conference on Machine Learning (ICML). — 2002. — P. 267–274.
10. Leviathan Y. Fast Inference from Transformers via Speculative Decoding / Y. Leviathan, M. Kalman, Y. Matias // Proceedings of the 40th International Conference on Machine Learning (ICML). — Honolulu, 2023. — P. 19274–19286.
11. Li Y. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty [Электронный ресурс] / Y. Li, F. Wei, C. Zhang, H. Zhang. — Режим доступа: <https://arxiv.org/abs/2401.15077> (дата обращения: 03.06.2026).
12. Puterman M. L. Markov Decision Processes: Discrete Stochastic Dynamic Programming / M. L. Puterman. — New York : John Wiley & Sons, 1994. — 649 p.
13. Qwen Team. Qwen2.5 Technical Report [Электронный ресурс] / Qwen Team. — Режим доступа: <https://arxiv.org/abs/2412.15115> (дата обращения: 03.06.2026).
14. Sutton R. S. Reinforcement Learning: An Introduction / R. S. Sutton, A. G. Barto. — 2nd ed. — Cambridge : MIT Press, 2018. — 552 p.
15. Svirschevski R. SpecExec: Massively Parallel Speculative Decoding for Interactive LLM Inference on Consumer Devices / R. Svirschevski, A. May, Z. Chen [et al.] // Advances in Neural Information Processing Systems (NeurIPS). — 2024.
16. Topkis D. M. Supermodularity and Complementarity / D. M. Topkis. — Princeton : Princeton University Press, 1998. — 272 p.
17. Vaswani A. Attention Is All You Need / A. Vaswani, N. Shazeer, N. Parmar [et al.] // Advances in Neural Information Processing Systems (NeurIPS). — 2017. — P. 5998–6008.

18. Yin M. A Theoretical Perspective for Speculative Decoding Algorithm / M. Yin, M. Chen, K. Huang, M. Wang [Электронный ресурс]. — Режим доступа: <https://arxiv.org/abs/2411.00841> (дата обращения: 03.06.2026).
19. Zheng L. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena [Электронный ресурс] / L. Zheng, W.-L. Chiang, Y. Sheng [et al.]. — Режим доступа: <https://arxiv.org/abs/2306.05685> (дата обращения: 03.06.2026).

## ПРИЛОЖЕНИЕ А. ДОКАЗАТЕЛЬСТВА ТЕОРЕМ

В приложении собраны доказательства утверждений, идеи которых приведены в разделах 1 и 2. Нумерация теорем сохранена; формулы приложения нумеруются (А.номер).

### А.1 Свойство точности спекулятивного декодирования

Утверждение. Последовательное применение правила (1.2) к  $\gamma$  черновым токенам, порождённым моделью  $q$ , даёт совместное распределение принятого префикса и замещающего токена, в точности совпадающее с распределением прямой авторегрессионной генерации из целевой модели  $p$ .

Доказательство ведётся индукцией по длине черновика  $\gamma$ . База ( $\gamma = 1$ ): один черновой токен принимается с вероятностью  $\min(1, p/q)$ , а при отклонении пересемплируется из нормированного остатка  $(p - q)_+$ ; прямая проверка показывает, что итоговое распределение токена равно  $p$ . Шаг индукции: пусть утверждение верно для длины  $\gamma - 1$ . Для первого токена результат правила (1.2) распределён по  $p$ ; при его принятии контекст обновляется, и к оставшимся  $\gamma - 1$  кандидатам применяется предположение индукции, дающее условное распределение целевой модели; при отклонении итерация завершается одним токеном из остатка. Совместное распределение записывается суммой по числу  $N$  подряд принятых токенов,

$$P(x_1, \dots, x_{N+1}) = \sum_k \Pr[N = k] p(x_1, \dots, x_{k+1}) \quad (\text{A.1})$$

где каждое слагаемое по теореме о произведении условных вероятностей и предположению индукции воспроизводит распределение  $p$  на соответствующей подпоследовательности. Марковость генерации обеспечивает независимость шага от контекста, что завершает доказательство.

### А.2 Теорема А (выборочная сложность оценщика переходов)

Доказывается оценка (2.10). По неравенству треугольника ошибка функции ценности разбивается на статистическую и смещения.

Статистическая часть: для каждой пары  $(s, a)$  эмпирическое распределение переходов есть нормированный счётчик, к которому по координатам применяется неравенство Хёффдинга; объединение оценок (union bound) по всем  $|S| \cdot |A|$  парам даёт логарифмический множитель  $\ln(2|S| \cdot |A| / \delta)$ . Перенос ошибки переходов на ошибку ценности использует  $\lambda$ -сжатие оператора Беллмана и вносит множитель  $1 / (1 - \lambda)^2$ : одна степень отвечает за горизонт, вторая — за распространение ошибки через неподвижную точку (стандартный приём, см. [1]).

Часть смещения: сглаживание Лапласа с параметром  $\alpha$  при  $n$  наблюдениях отклоняет оценку от несглаженной не более чем на  $\alpha / (n + \alpha)$  по координатам; суммирование по носителю и по парам  $(s, a)$  и распространение через сжатие дают аддитивный член  $\alpha |S| R / ((1 - \lambda)(n_{\min} + \alpha))$ . Сложение двух частей и даёт правую часть (2.10) с вероятностью не менее  $1 - \delta$ . Бэунд не туг по абсолютной величине, но устанавливает скорость сходимости порядка  $1 / \sqrt{n_{\min}}$  с убывающим по  $n_{\min}$  смещением, чего достаточно для обоснования табличного пайплайна.

### **A.3 Теорема В (Беллман-инвариантность аддитивного штрафа)**

Пусть к награде добавлен штраф  $-g(s)$ , зависящий только от состояния. Уравнение оптимальности для изменённого MDP принимает вид

$$V_g^i(s) = -g(s) + \max_{a \in A} \left[ r(s, a) + \lambda \sum_{s'} P(s' | s, a) V_g^i(s') \right] \quad (\text{A.2})$$

Слагаемое  $-g(s)$  не зависит от действия и выносится за знак максимума, поэтому  $\arg\max$  по  $a$  —  $a$  с ним и оптимальная политика — совпадает с политикой исходного MDP; функция ценности отличается лишь на дисконтированную сумму штрафов вдоль траектории. Следовательно, штраф качества способна изменить только связанная с действием форма, что и обосновывает конструкцию награды (2.8). Аддитивность сохраняет  $\lambda$ -сжатие оператора, тогда как мультипликативная форма его разрушала.



#### **А.4 Теорема С (оценка субоптимальности каскада)**

Доказательство опирается на лемму о разности производительностей [9], выражающую разрыв ценности двух политик через стационарное среднее преимущества:

$$V_{joint} - V_{cascade} = \frac{1}{1 - \lambda} E_{s \sim \mu_J^*} [A^{\pi_c}(s, \pi_J(s))] \quad (A.3)$$

Вне множества В нарушения условия С4 каскадная политика локально оптимальна, и преимущество неположительно; на В оно по модулю не превосходит  $2R_{max} / (1 - \lambda)$ . Разбивая ожидание по индикатору В и оценивая часть на В сверху, получаем оценку (2.11), линейную по стационарной массе нарушения  $\mu_J^*(B)$ . Честная оговорка: эмпирически  $\mu_J^*(B) \approx 0,90$  (см. приложение В), поэтому правая часть (2.11) оценивается величиной порядка нескольких сотен и как абсолютная оценка вакуумна; содержательный результат даёт теорема D.

#### **А.5 Теорема D (точный разрыв ценности)**

Равенство (A.3) выполнено точно, а не только как оценка сверху. Поэтому величина разрыва ценности определяется не размером множества В, а средним преимуществом каскадной политики на нём. Эмпирическое усреднение этого преимущества пренебрежимо мало (порядка  $6 \cdot 10^{-5}$  на паре 14В/0.5В и  $10^{-3}$  на паре 7В/1.5В), поэтому, несмотря на повсеместное нарушение С4, точный стационарно-взвешенный разрыв мал (+0,005 и +0,10 соответственно). Это и объясняет наблюдаемое практическое совпадение совместной и каскадной политик: нарушение С4 повсеместно, но доброкачественно.

#### **А.6 Теоремы 2.3 и 2.4**

Теорема 2.3 (слабое доминирование). Любая каскадная политика реализуема и в совместном пространстве, то есть  $\Pi_{cascade} \subset \Pi_{joint}$ . Оптимум по большему множеству политик не меньше оптимума по меньшему,

поэтому  $V_{\text{joint}}(s) \geq V_{\text{cascade}}(s)$  во всех состояниях  $s$ . Строгого доминирования это рассуждение не даёт, и эмпирически (раздел 4) разрыв статистически неотличим от нуля.

Теорема 2.4 (скаляризация Парето). Множество достижимых пар  $(\eta, D)$  по всем стохастическим стационарным политикам выпукло: рандомизация политик с весами  $w_i$  даёт пару

$$(\eta, D) = \sum_i w_i (\eta_i, D_i) \quad (\text{A.4})$$

то есть выпуклую комбинацию операционных характеристик. Линейный функционал  $\eta - \kappa D$  достигает максимума на выпуклом множестве в опорной точке его верхней границы — Парето-фронта — с наклоном опорной прямой  $\kappa$ . Перебор  $\kappa \geq 0$  обходит фронт, что и утверждает теорема.

## ПРИЛОЖЕНИЕ Б. АРХИТЕКТУРА РЕПОЗИТОРИЯ И ЛИСТИНГИ

Программный комплекс образован двумя стеками — историческим `sp_samp/` с эталонными реализациями и тематическим `jointadaspec/` (подпакеты `core`, `mdp`, `inference`, `baselines`, `metrics`, `analysis`, `utils`; таблица 3.1). Поток данных линейен: стадия 01 собирает трейсы, 02 оценивает MDP и решает его, 03 выполняет бенчмарк, 04 проверяет условия (рисунок 3.1). Ниже приведены ключевые фрагменты реализации.

Листинг Б.1 — Поля одношаговой трейс-записи (`jointadaspec/mdp/traces.py`)

```
record = {
    "state_index": state_idx,      # дискретный индекс  $s = (i_H, i_K, k)$ 
    "action_index": action_idx,    # индекс действия ( $a_{length}, T$ )
    "k": k,                        # позиция в черновике
    "accepted": int(accepted),     # принят ли токен
    "proposed": int(proposed),     # предложен ли токен черновиком
    "reward": reward,              # мгновенная награда  $r(s, a)$ 
    "next_state_index": next_idx,  # следующее состояние  $s'$ 
}
```

Листинг Б.2 — Оценка параметров MDP со сглаживанием

(`jointadaspec/mdp/estimation.py`)

```
@dataclass
class EstimatedMDP:
    transitions: sparse.csr_matrix  #  $(|S| * |A|) \times |S|$ , ~1% ненулевых
    rewards: np.ndarray             #  $(|S|, |A|)$ 
    visit_counts: np.ndarray        #  $(|S|, |A|)$ 

def estimate_mdp_parameters(traces_path, config):
    # накопить  $N(s,a,s')$  и суммы наград из Parquet-трейсов;
    # при  $visits \geq nu\_min$  — прямая оценка, иначе — сглаживание
    # Лапласа с  $alpha\_smooth$  на локальном носителе соседних состояний;
    #  $rewards[s,a] = reward\_sums[s,a] / visits$ 
    return EstimatedMDP(transitions, rewards, visit_counts)
```

Листинг Б.3 — Шаг онлайн-декодера (`jointadaspec/inference/jointadaspec.py`)

```
H = entropy(q_probs); K = self.K_prev          # признаки состояния
action_length, threshold = self.policy.get_action(H=H, K=K, k=k)
if action_length == "stop":
    self._verify_block(...)                    # нечёткая
    верификация, T
else:
    k = min(k + 1, self.policy.config.gamma_max) # продолжить черновик
    self.K_prev = kl_divergence(q_probs, p_probs) # задержанное
    обновление K
```

## ПРИЛОЖЕНИЕ В. ДОПОЛНИТЕЛЬНЫЕ ЭКСПЕРИМЕНТАЛЬНЫЕ ДАННЫЕ

Приложение содержит вспомогательные данные, дополняющие раздел 4: интерпретацию выученной политики, проверку теоретических условий и точные величины теорем С и D.

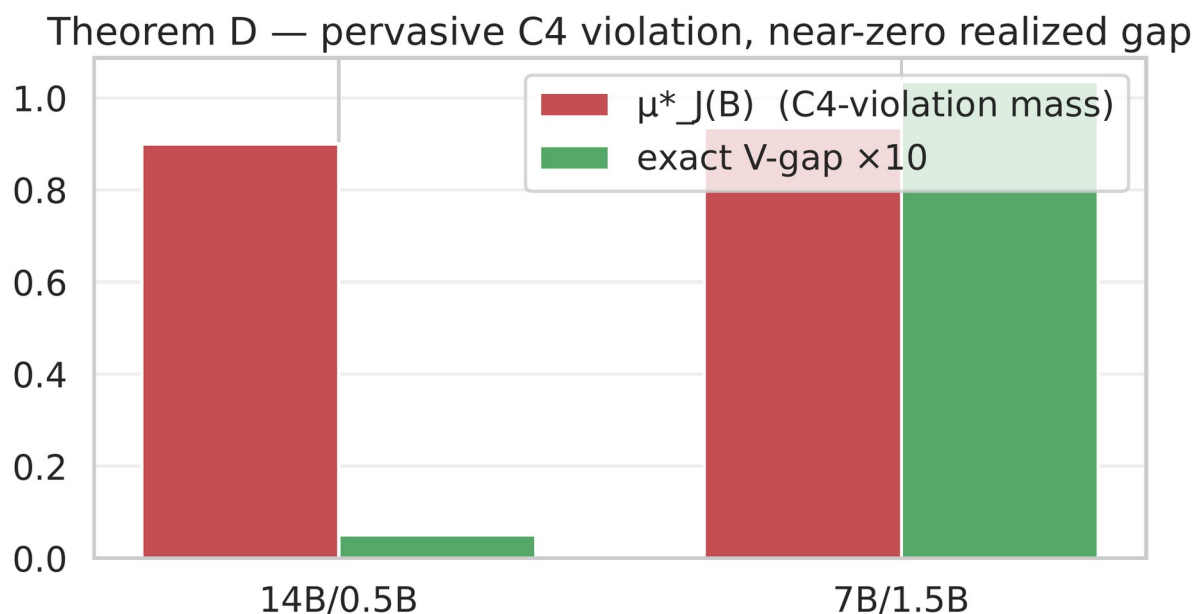


Рисунок В.1 — Распределение преимущества каскадной политики на множестве нарушения условия С4: масса сосредоточена около нуля (теорема D)

Learned joint policy on 14B/0.5B — mild adaptivity in qualitatively sensible directions

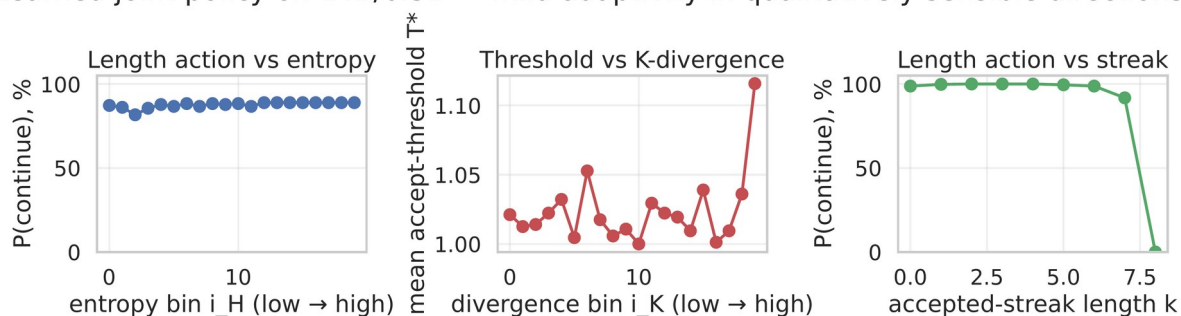


Рисунок В.2 — Интерпретация выученной политики: вероятность продолжения черновика и средний выбранный порог в зависимости от признаков состояния

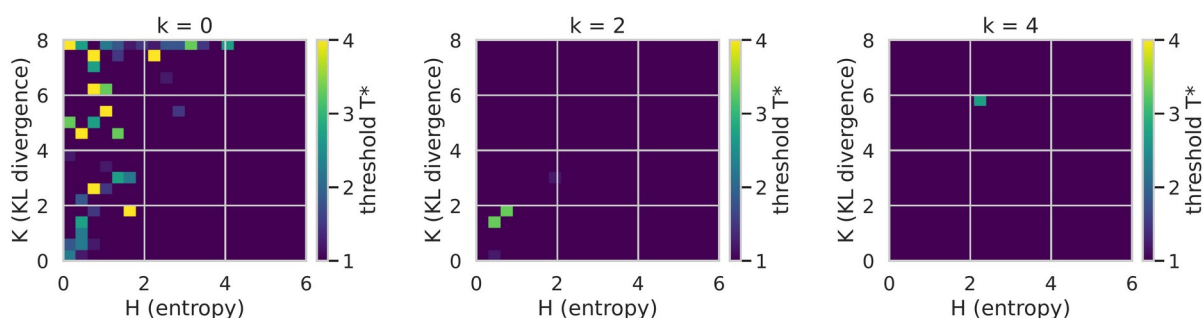


Рисунок В.3 — Пороговая поверхность выученной политики в координатах (энтропия  $H$ , дивергенция  $K$ )

Точные величины, лежащие в основе теорем С и D, вычислены по сошедшимся политикам обеих пар моделей (скрипт `scripts/analyze_theorem_c_gap.py`) и сведены в таблице В.1.

Таблица В.1 — Точные величины теорем С и D по обеим парам моделей

| Величина                                                       | 14B/0.5B      | 7B/1.5B       |
|----------------------------------------------------------------|---------------|---------------|
| Доля состояний с нарушением С4                                 | 0,89          | 0,89          |
| Стационарная масса $\mu^*_J(B)$                                | 0,90          | 0,93          |
| Оценка теоремы С (правая часть)                                | $\approx 360$ | $\approx 374$ |
| Точный разрыв ценности $V_{\text{joint}} - V_{\text{cascade}}$ | +0,005        | +0,10         |
| Доля состояний со слабым доминированием                        | 100 %         | 100 %         |

Таблица показывает ключевое: при почти повсеместном нарушении условия С4 (масса  $\approx 0,9$ ) оценка теоремы С вакуумна, однако точный разрыв ценности мал, а слабое доминирование (теорема 2.3) выполнено на всех состояниях. Развёртка по множителю  $k$  (рисунок 4.4) подтверждает устойчивость равенства совместной и каскадной политик ко всему диапазону трейд-офф-параметра.

## ПРИЛОЖЕНИЕ Г. МАНИФЕСТЫ ВОСПРОИЗВОДИМОСТИ

Каждый содержательный прогон сопровождается машинно-генерируемым манифестом в каталоге `reports/manifests`. Манифест фиксирует окружение (версии библиотек, ускоритель), состояние репозитория (хеш коммита Git и признак незакоммиченных изменений), полную разрешённую конфигурацию Hydra, список начальных значений генераторов и контрольные суммы SHA256 артефактов трейсов и политики. Этого достаточно, чтобы однозначно восстановить условия прогона. Сокращённый пример приведён в листинге Г.1.

Листинг Г.1 — Фрагмент манифеста прогона 14B/0.5B (lock, 2026-05-14)

```
{
  "generated_at": "2026-05-13T19:40:46Z",
  "git_sha": "e1b32ea7...", "git_branch": "main", "git_dirty": true,
  "nvidia_smi": "NVIDIA GeForce RTX 5090, 32607 MiB",
  "python": "3.12.3",
  "torch_version": "2.9.1+cu128", "transformers_version": "4.57.3",
  "scipy_version": "1.17.0", "numpy_version": "2.4.2",
  "seed_list": [42, 43, 44],
  "policy_path": "outputs/.../02_solve/policy.npz",
  "policy_npz_sha256": "f0f1e09c0b41cfc9...",
  "hydra_config": {"N_max": 6.0, "K_max": 8.0, "gamma_max": 8,
                  "N_H": 20, "N_K": 20, "lambda_discount": 0.99,
                  "kappa": 1.0, "alpha_smooth": 1.0, "nu_min": 5}
}
```

Манифест сохраняется автоматически стадией `scripts/05_write_manifest.py` и привязывает результаты раздела 4 к конкретному состоянию кода и данных. Отдельные вендорные ядра CUDA сохраняют предупреждение о неполной воспроизводимости; в этом случае первичной записью считаются именно манифест и список начальных значений, а малый числовой дрейф признаётся допустимым.